

ArrowFlow: Training Ranking Filters with Position Votes

Ozgur Yilmaz*

Department of Artificial Intelligence

Adana Alparslan Turkes Science and Technology University

Adana 01250, Turkey

Abstract

ArrowFlow is a classifier whose layers work with rankings. Each layer holds learned rankings, the ranking filters, and outputs them sorted by footrule distance to its input. Training examples vote on where a filter’s items should go. A weighted Borda count merges the votes, with no derivative computed. The position displacements (motions) returned by the output layer’s votes train the hidden layers. A fixed encoder sorts projected features, and a nearest-neighbor vote reads the last hidden ranking. Under nested cross-validation on seventeen datasets, seven used during development and ten chosen by prespecified rules, training the ranking layers improves performance. Scrambled, the motions leave the network less accurate on most datasets than never moving its hidden filters. Turned into target scores, motions of the same kind also train a neural encoder network by gradient steps, without differentiating the sort. As a classifier, the trained encoder network is more accurate than the same network untrained on all 17, significantly on 11. Swapped into ArrowFlow, it raises mean accuracy on 11 datasets, significantly on one, and lowers it significantly on none. Training a neural network via discrete motion signals from a ranking layer is novel and one of the most important contributions of the paper. Counting the filter’s current order as one more voter, the update rule violates independence of irrelevant alternatives by Arrow’s theorem under a mild weight condition. So a filter’s order of two items can change when a third moves.

Keywords: ranking filters; permutation learning; Borda count; Arrow’s impossibility theorem; Spearman’s footrule; nearest-neighbor classification; alternatives to back-propagation; target propagation

1 Introduction

Most neural networks store their inputs and their learned parameters as numbers, and they learn by following gradients. This paper asks whether a network can instead compute with

*Correspondence: ozguryilmaz@atu.edu.tr; <https://scholar.google.com/citations?user=AIJWYCAAAAAJ>

rankings, ordered lists of items, and learn by counting votes. It also asks whether the signal of those votes can train an ordinary neural network as well. Codes built from orders appear in neuroscience and in similarity search (Section 2). Here the network learns the orders that it compares its inputs with.

ArrowFlow builds such a network from four ingredients:

- **Data points** are rankings. A ranking is a permutation of a fixed set of items, the vocabulary.
- **Learned filters** are rankings of the same vocabulary. Each *ranking filter* is a permutation prototype, a learned ranking that an input is compared with.
- **Similarity** is Spearman’s footrule distance. It counts the total number of positions by which the items of one ranking must move to match another [Diaconis and Graham, 1977].
- **Learning** reorders the items of a filter by counting votes for their positions. No derivative is computed.

An encoder turns a real-valued example into a ranking (Figure 1). It expands, standardizes and projects the features, and then it sorts the projected scores. A *ranking layer* holds a bank of ranking filters and outputs the bank in order of distance to its input, nearest filter first. That output is itself a ranking, so the next layer can read it. To predict, ArrowFlow compares an example’s last hidden ranking with those of the training examples, and the nearest ones vote on the class. We call this step the nearest-neighbor readout. Several *views*, each an encoder with its own trained network, then vote on the final class.

Training is the main contribution of this paper: a layer of ranking filters are adjusted based on motion signals that are coming from higher layers. It is similar to backpropagation, but it is on ranking filters. It is the first known method to train a hierarchical network through discrete signals that are not derivatives. During training, the network gets one more layer, an output layer with one filter per class. When this layer assigns an example to the wrong class, the filter of the true class votes to move toward the example’s last hidden ranking. The class filter and the example’s ranking are both rankings of the same hidden filters. The signed differences between the positions that they give each hidden filter are called *motions*, and the motions travel back down the network. If the class filter ranks a hidden filter earlier than the example’s ranking does, that hidden filter votes to move toward its input. If the class filter ranks it later, the filter votes to move away. Nothing is differentiated along the way. Still, whenever the class filter votes, the motions are exactly half the gradient of the squared distance between the two rankings’ positions, taken with respect to the positions of the hidden filters in the example’s ranking. A hidden filter’s vote then tends to move its position the way a gradient step would. So the vote stands in for the missing derivative of the sort, as a straight-through estimator does [Bengio et al., 2013]. Such an estimator trains through a discrete step by replacing the step’s derivative with a simple surrogate (Section S1.8).

Every update of a ranking filter merges the rankings of a training batch into one, so every update is an election. Arrow’s impossibility theorem [Arrow, 1951, 1963] says that no way of merging rankings has every property one might want. With three or more alternatives, no rule that turns every profile of rankings into one ranking satisfies three conditions at once:

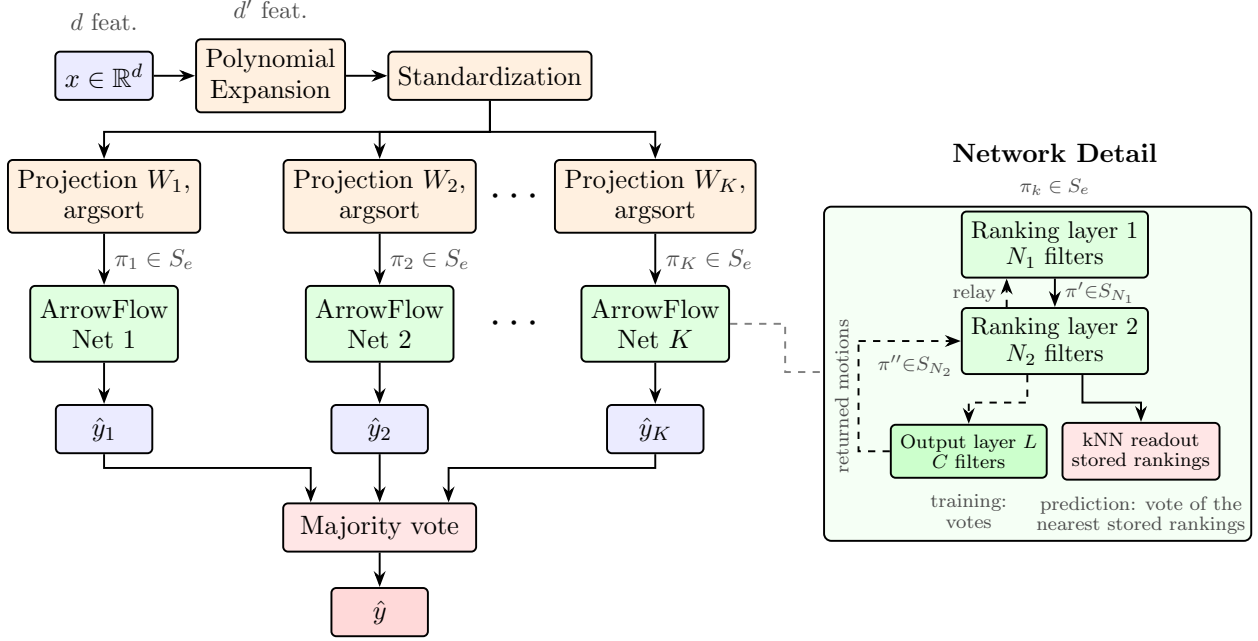


Figure 1: **ArrowFlow pipeline**. A real-valued input x is encoded by polynomial expansion of degree p , standardization, a projection and argsort. The K views use different projections W_k , $k = 1, \dots, K$, cycling through target-aware, random and calibrated strategies, so their networks receive different input rankings. Each network (detail, right) is a stack of ranking layers, and every layer ranks its ranking filters by Spearman’s footrule distance to its input ranking. In training, an output layer holds one filter per class, and the filter of the example’s class votes to move toward the last hidden ranking. The motions this vote returns train the last hidden layer, which relays signed motions to the preceding layer (dashed arrows). To predict, a nearest-neighbor (kNN) readout compares the last hidden ranking with the stored hidden rankings of the training examples. The K predictions are combined by majority vote. S_n denotes the set of rankings of n items.

Pareto efficiency, independence of irrelevant alternatives and non-dictatorship. This is a lens to look at the update rule. It gives us a simple perspective on the context dependence of a filter’s order of two items, which is essential for nonlinearity in the network. Independence asks that the merged order of two items depend only on how the voters order those two items. Our update is a weighted Borda count, which orders items by their weighted sums of positions, and the filter’s current order counts as one more voter. The rule is Pareto efficient for any positive weights. Under a mild condition on the weights it is also non-dictatorial, so it must break independence. The condition says that the votes of a batch weigh enough that they can move the filter. In ArrowFlow’s layers, an update that holds the current order fixed is not Pareto efficient, so the theorem does not apply to it. It breaks independence too, by a direct construction, whenever its votes can move the filter (Section S1.1). So a filter’s order of two items can change when the training rankings move a third.

We use the theorem as a design lens, and it promises nothing about accuracy. It shows that this context dependence lives in the learning rule. And the voting reading names three stages of the network whose rules can be exchanged. The name joins *Arrow*, for the vote inside every filter update, and *Flow*, for the rankings that pass from layer to layer.

What this paper shows. We did not build ArrowFlow to be the most accurate classifier, and it is not one. With all settings tuned on the training data alone, it trails the best tuned classical model on all seventeen tabular datasets we test (Table 3). Its average rank is fourth among the six tuned models, between random forest and gradient boosting, and fifth on the ten further datasets alone. What the paper shows instead is that votes, without computing any derivative, can train ranking layers, and that the motions they return can train an ordinary neural network:

1. *Training helps.* The trained network is more accurate than an untrained ArrowFlow on 16 of the 17 datasets, although the untrained version comes within one percentage point on 12 of the 17 (Section 7.3).
2. *The training signal carries information.* Suppose we scramble the motions but keep their sizes. The network then ends up less accurate than if its hidden filters had never moved. This happens on 16 of the 17 datasets when each motion reaches the wrong filter, and on 14 of the 17 when its direction is random. A post hoc test across the datasets, one defined after the results were known, supports both (Section 7.5).
3. *Motions also train a real-valued network.* Motions of the same kind can be turned into target values for the scores that a small encoder network sorts. They then train that network with a standard gradient-based optimizer, although the sort between the two is never differentiated. Other methods also train through an exact discrete step (Section 2). The trained encoder network is more accurate than the same network untrained on all 17 datasets, significantly on 11. Swapped into ArrowFlow for the fixed encoder, it raises the mean accuracy on 11 of the 17 datasets, significantly on balance-scale, and lowers it significantly on none (Section 7.7). The conversion of motions into target scores and its settings were developed on single splits of twelve of the seventeen datasets (Section S6.2).
4. *The contribution is the learning rule, and its limits are clear.* A support vector classifier

with a fixed Kendall kernel, a standard similarity between rankings, is more accurate than ArrowFlow on 14 of the 17 datasets when both read rankings from the same encoder family. Read by that classifier, trained hidden rankings are no better than untrained ones. So training improves the neighborhoods of the nearest-neighbor readout, but the representation read by a strong fixed readout shows no gain. A second hidden layer did not help (Section 7.6).

Sections 2 to 6 cover related work, the network, the Arrow lens and the theory, and Section 7 reports the experiments. Section 8 discusses the results, and Section 9 concludes. The supplement, Sections S1 to S6, holds the proofs, the method details and protocol, the complete results, the ablations, the reproducibility records and the details of the trained encoder.

2 Related Work

ArrowFlow touches seven lines of work. The first studies ordinal codes and distances between rankings, and the second studies voting rules that combine rankings. The third covers prototype methods, and the fourth covers kernels, metric learning and symbolic vectors. The fifth covers alternatives to backpropagation, and the sixth covers ways to train through a sort. The seventh supplies the tools that ArrowFlow’s pipeline borrows. For each line we say what it does and where ArrowFlow departs from it.

Ordinal codes and distances between rankings. The idea that order alone can carry information is old. Rank-order coding holds that the order in which neurons fire carries information about the stimulus [Thorpe and Gautrais, 1996, Gautrais and Thorpe, 1998, Thorpe et al., 2001], and Bonilla et al. [2022] compare it with codes based on the time of the first spike. The rank transform of nonparametric statistics [Conover and Iman, 1981] also keeps only order. But it ranks one variable across observations, whereas top-scoring-pair classifiers compare pairs of features within one example [Geman et al., 2004]. Rank-ordered autoencoders order hidden activations and learn sparse codes through progressive reconstruction [Bertens, 2016]. ArrowFlow instead feeds an ordinal code into layers of learned ranking filters and compares rankings by Spearman’s footrule. Diaconis and Graham [1977] show that the footrule stays within a factor of two of Kendall’s distance, the number of item pairs that two rankings order differently [Kendall, 1938]. Fagin et al. [2003] and Fagin et al. [2006] extend both distances to top- k lists and to partial rankings. Learning to rank [Burges et al., 2005, Burges, 2010] fits a real-valued scoring function whose output is an ordering. In ArrowFlow, the inputs and the parameters are themselves orderings.

Voting rules and rank aggregation. Combining many rankings into one is the subject of social choice theory. Arrow’s theorem [Arrow, 1951, 1963] limits every rule that turns a profile of rankings, one per voter, into one ranking. Positional rules, such as the Borda count [Young, 1974], give each item points for its position on each ballot. Distance rules, such as the Kemeny median [Kemeny, 1959] and the footrule median, pick the ranking with the smallest total distance to the ballots. Both kinds have been compared in rank aggregation for metasearch [Dwork et al., 2001]. The Mallows model [Mallows, 1957] is a probability distribution over rankings that is centered on one modal ranking. Conitzer and Sandholm

[2005] show which voting rules are maximum-likelihood estimators under which noise models, so choosing a rule also chooses what it estimates. Under Mallows noise, positional rules such as the Borda count recover the modal ranking as the number of votes grows [Caragiannis et al., 2013]. The Borda count also combines the class rankings of several classifiers [Ho et al., 1994, van Erp and Schomaker, 2000]. ArrowFlow applies a weighted Borda count inside a learning system. Section 4 states what Arrow’s theorem implies for the network and what it does not.

Prototypes and reference objects. Many classifiers predict from distances to learned reference patterns. Learning vector quantization (LVQ) and self-organizing maps work this way [Kohonen, 1990], and so do generalized LVQ [Sato and Yamada, 1995, Hammer et al., 2014, Nebel et al., 2015] and adaptive resonance theory [Grossberg, 1987]. Diao et al. [2021] combine random-feature encodings with generalized LVQ and connect the result to hyperdimensional classifiers. ArrowFlow’s output layer, which drives training, is a prototype classifier of this kind, with permutations as prototypes. Describing an object by its distances to reference objects is also the dissimilarity representation of Pękalska and Duin [2005]. Metric-space search goes one step further. It represents an object by the order of the reference objects by their distance to it, and it compares such orders with rank distances [Chávez et al., 2008, Amato and Savino, 2008]. An untrained ranking layer read by a footrule nearest-neighbor vote is such a permutation index, with random rankings instead of data points as references. ArrowFlow learns the reference permutations and stacks the construction in layers. Random-weight networks fix random hidden units and train only a readout. Random vector functional-link networks [Pao et al., 1994] and extreme learning machines [Huang et al., 2006, 2012] are examples, and random features approximate kernel machines in the same spirit [Rahimi and Recht, 2007]. An untrained ArrowFlow is a permutation analog of these models.

Kernels, metric learning and symbolic vectors. Other methods also classify permutation-valued inputs, but they do not learn the permutations themselves. Kernels on permutations are the established supervised route. Jiao and Vert [2015, 2018] classify with Kendall and Mallows kernels, including numerical data converted to rankings, and they learn a decision function from pairwise similarities. Supervised metric learning fits a representation for a nearest-neighbor classifier [Goldberger et al., 2004, Weinberger and Saul, 2009]. ArrowFlow instead trains its permutations against one prototype per class. Vector symbolic architectures and hyperdimensional computing (HDC) represent structured objects in high-dimensional distributed vectors. Holographic reduced representations bind and superpose vectors with circular convolution [Plate, 1995]. Kanerva’s construction labels an item’s position before summation so that order survives [Kanerva, 2009], and ordered position codes serve biosignal classification [Rahimi et al., 2019]. The surveys of Kleyko et al. [2022, 2023] map the field. In these systems the state is distributed and algebraic. The codes are still related to ArrowFlow’s. A thermometer code writes position p as p ones followed by zeros, and the footrule equals the Hamming distance between the thermometer codes of the positions. But ArrowFlow stores explicit permutations and updates them by position votes. Learned hyperdimensional classifiers train their class vectors [Duan et al.,

2022, Smets et al., 2025], and Vergés et al. [2025] review and compare such classifiers. So the hyperdimensional control of Section S3.5, an untrained classifier built from fixed random codes of items and positions, represents only its own construction. It says nothing about learned hyperdimensional classifiers. Discrete encoders can be trained with native integer and binary operations, with reported gains over untrained ones [Kirby et al., 2026].

Alternatives to backpropagation. The forward-forward algorithm [Hinton, 2022], target propagation [Bengio, 2014] and its difference form [Lee et al., 2015], and feedback alignment [Lillicrap et al., 2016] avoid sending exact gradients back through a network, but they keep real-valued weights. ArrowFlow replaces the weights by permutations and updates them with position votes. Each batch update exactly minimizes a squared position objective of one filter (Section S1.8), but no loss of the whole network is minimized. The output layer’s vote only attracts the filter of the true class, whereas LVQ1 and LVQ2.1 also push a prototype of a wrong class away [Kohonen, 1990]. A hidden layer computes the signal for the layer below from the permutations of its own selected filters. This is the analog of weight transport, the reuse of the forward weights to send errors back, which feedback alignment avoids. We do not propose the rule as a model of learning in the brain.

Training through a sort. Differentiable sorting replaces the permutation with a smooth approximation so that gradients can pass through it. NeuralSort [Grover et al., 2019], SoftSort [Prillo and Eisenschlos, 2020], projection onto the permutahedron [Blondel et al., 2020b], optimal transport [Cuturi et al., 2019] and Gumbel-Sinkhorn networks [Mena et al., 2018], built on Sinkhorn normalization [Sinkhorn and Knopp, 1967], all work this way. ArrowFlow keeps the sort exact and never differentiates through it. Its trained encoder (Section 5.5) also learns through an exact sort, by turning the motions into target scores in the spirit of target propagation [Bengio, 2014, Lee et al., 2015]. Other methods also keep a discrete step exact. The straight-through estimator copies a gradient across the step as if the step were the identity [Bengio et al., 2013, Sahoo et al., 2023]. Blackbox differentiation interpolates the loss behind the solver, using a second call to the solver [Vlastelica et al., 2020, Rolínek et al., 2020]. Perturbed optimizers average the solver’s output over random perturbations of its input [Berthet et al., 2020], and implicit maximum-likelihood estimation builds a target distribution for it [Niepert et al., 2021]. LambdaRank sets score gradients from desired rank changes [Burgess et al., 2006]. The structured perceptron and its generalization, the Fenchel-Young losses, train a scoring model on the exact output of a solver [Collins, 2002, Blondel et al., 2020a]. Target propagation for hard-threshold units sets discrete targets for the hidden units and trains each unit toward them [Bengio, 2014, Friesen and Domingos, 2018]. The trained encoder’s targets come from motions toward a learned class filter and away from the nearest wrong one, and the example’s own sorted scores turn them back into scores.

Random projections and ensembles. ArrowFlow’s fixed encoder rests on random projection. The known guarantees on preserved distances [Johnson and Lindenstrauss, 1984, Achlioptas, 2003] and preserved margins [Arriaga and Vempala, 2006] concern the projected vector before sorting. The order of two Gaussian-projected coordinates is a random-

hyperplane bit [Charikar, 2002], the sign of a random linear function of the input. The order of Gaussian projections also underlies the rank-order hashes for cosine similarity of Eshghi and Rajaram [2008], and it relates to winner-take-all hashing [Yagnik et al., 2011]. Section 6 states the resulting angle property. The multi-view ensemble is related to random-projection ensemble classification [Cannings and Samworth, 2017], although ArrowFlow neither selects its projections nor thresholds the vote. It stands in the tradition of bagging [Breiman, 1996], boosting [Schapire, 1990], ensemble diversity [Dietterich, 2000, Kuncheva and Whitaker, 2003] and multi-view learning [Xu et al., 2013].

3 Materials and Methods: The Ranking Layer

The ranking layer is the basic unit of ArrowFlow. It receives a ranking and measures how far that ranking lies from each ranking in a bank of learned rankings. Then it outputs the bank sorted by that distance. Learning reorders the learned rankings by collecting votes for where their items should sit, and no derivative is computed anywhere in the ranking layers. This section defines the layer, the training rule and the readout exactly.

3.1 Notation

A *ranking* of a vocabulary of V items is a permutation π that lists every item exactly once. We write $\pi[p]$ for the item at position $p \in \{1, \dots, V\}$ and $\text{pos}_\pi(v)$ for the position of item v . The two maps are inverses, and $\text{rev}(\pi)$ is π read backwards. Worked examples list items by position, so $\pi = [C, A, E, B, D]$ means $\pi[1] = C$. Every item carries a distinct integer identifier (ID), and every sort in the method breaks exact ties by ascending ID (Section S2.1). Table 1 collects the symbols used from here on.

3.2 Ranking Filters and Motions

A layer compares its input with each of its filters by asking how far the items must move. It holds N *ranking filters* $r_1, \dots, r_N$. A ranking filter is a permutation prototype, a learned permutation of the layer’s vocabulary that the layer compares with its input.

The comparison rests on the *motion* of one ranking π toward another ranking τ of the same vocabulary,

$$m_{\pi \rightarrow \tau}[p] = \text{pos}_\tau(\pi[p]) - p, \quad p = 1, \dots, V, \quad (1)$$

the signed number of positions that the item at position p of π must move to reach its position in τ . A positive motion points toward the end of the ranking, and a negative one toward the front. For an input π_x and a filter r_j the motion is $m_j = m_{\pi_x \rightarrow r_j}$. Summing the absolute motions gives Spearman’s footrule distance [Diaconis and Graham, 1977],

$$d_F(\pi, \tau) = \sum_{p=1}^V |m_{\pi \rightarrow \tau}[p]| = \sum_v |\text{pos}_\pi(v) - \text{pos}_\tau(v)|. \quad (2)$$

The footrule is symmetric and takes even whole-number values from 0 to $\lfloor V^2/2 \rfloor$. It is the only distance the network computes, although the batch update minimizes squared position

Table 1: **Notation.** The layer superscript (ℓ) is dropped when a single layer is discussed.

Symbol	Meaning
$\pi, \pi[p], \text{pos}_\pi(v), \text{rev}(\pi)$	a ranking; its item at position p ; the position of item v ; the reversed ranking
π_x, π'	the encoded input ranking; the output ranking of a layer
V	vocabulary size of a layer, the number of items it ranks
ℓ, L	layer index and output layer; layer ℓ has vocabulary size V_ℓ and N_ℓ filters, and $V_{\ell+1} = N_\ell$
e, C	encoded vocabulary size, $V_1 = e$; number of classes, $N_L = C$
r_j, N	ranking filter j and the number of filters in a layer
$m_{\pi \rightarrow \tau}, m_j$	motion of π toward τ , (1); $m_j = m_{\pi_x \rightarrow r_j}$
$\mathcal{M}^{(\ell)}, \mathcal{M}_j^{(\ell)}$	signal list passed to layer ℓ , sorted by $(- \cdot , \text{ID})$; the entry of filter j
d_F, D_j	footrule distance, (2); $D_j = d_F(\pi_x, r_j)$
$\pi^{(\ell)}, \hat{y}$	output ranking of layer ℓ , with $\pi^{(0)} = \pi_x$; class $\hat{y} = \pi^{(L)}[1]$ of the prototype readout
$A_j, A_j[p, k], a, s_p$	accumulator of filter j (row p is the item at position p , column k a position); signed vote weight; mean voted position of row p
η, ρ, c, p_c	learning rate, selected fraction, signal scale, acceptance probability of a correct example
w, g	sample weight (one throughout this paper); gate value in $\{0, 1\}$
T, t, B, ν	number of iterations, iteration index, batch size, validation fraction
$u, \tilde{u}$	mean relayed motion and the scaled signal passed to the layer below, (6)
$h, \tilde{h}, q_p, m_y, m_w$	for the encoder network of Section 5.5: its scores, sorted in decreasing order into $\pi_x = \text{argsort}_\downarrow(h)$; their targets; the target position of the coordinate at position p ; the motions m_j for $j = y$, the true class, and $j = w$, the nearest wrong class

differences (Section 6.4). Figure 2 shows the motions of one input toward three filters and the distances $D_j = d_F(\pi_x, r_j)$.

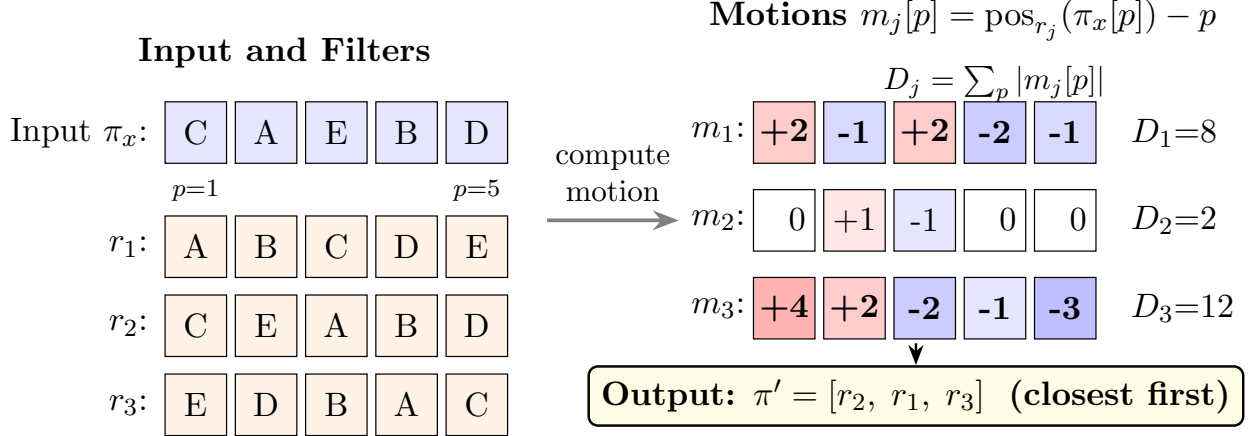


Figure 2: **Ranking layer forward pass.** The input ranking $\pi_x = [C, A, E, B, D]$ is compared with three ranking filters r_1, r_2, r_3 . The motion $m_j[p] = \text{pos}_{r_j}(\pi_x[p]) - p$ is the signed displacement of the item at position p toward its position in the filter (red: positive, toward the end of the ranking; blue: negative, toward the front). The footrule distance $D_j = \sum_p |m_j[p]|$ totals the displacements. The filters are ranked by distance, and r_2 (distance 2) is closest. The output ranking π' is the input of the next layer.

3.3 Forward Pass

A layer’s output is simply its filters, sorted from nearest to farthest, as in neural gas [Martinetz et al., 1993]. The output π' lists the filter IDs sorted by (D_j, j) , closest first, with ties going to the lower ID. This output is a ranking of a new vocabulary, the N filter IDs, and it is the next layer’s input.

A network stacks L ranking layers. Layer ℓ holds N_ℓ filters over V_ℓ items. The first layer ranks the $V_1 = e$ items of the encoded input (Section 5), and every later layer ranks the filters of the layer below, so $V_{\ell+1} = N_\ell$. We write $\pi^{(0)} = \pi_x$ and $\pi^{(\ell)}$ for the output of layer ℓ . The output layer L holds $N_L = C$ filters, one per class, and its output $\pi^{(L)}$ is the *class ranking*. The *prototype readout* takes the class $\hat{y} = \pi^{(L)}[1]$, whose filter is nearest to the last hidden ranking $\pi^{(L-1)}$. The output layer only serves training. It drives the updates and makes none of ArrowFlow’s predictions (Section 3.6, Table 2).

3.4 Position Votes and the Accumulator

Learning reorders a filter by counting votes for where each of its items should sit. Each filter r_j owns an *accumulator* $A_j \in \mathbb{R}^{V \times V}$, a table that collects these votes. Row p belongs to the item $r_j[p]$ currently at position p , and column k is a position. Between updates A_j is the identity matrix, which gives every item one prior vote for its current position.

A training example casts position votes with a signed weight a toward a target ranking τ . A positive vote pulls the filter toward τ . A negative vote pulls it toward the reversal of

(a) Accumulator A_j before the batchcolumn k = position

	1	2	3	4
$p=1$: A	1	0	0	0
$p=2$: B	0	1	0	0
$p=3$: C	0	0	1	0
$p=4$: D	0	0	0	1

row p : item at position p **(b) Votes from one batch**a vote of weight a adds $|a|$ to the column of each item's target position

vote 1 ($a_1 = +1$)	C	A	D	B	$\Rightarrow +1$ at (A, 2), (B, 4), (C, 1), (D, 3)
vote 2 ($a_2 = -0.5$)	D	B	C	A	$a_2 < 0$: the target is reversed
reversed ($a_2 < 0$)	A	C	B	D	$\Rightarrow +0.5$ at (A, 1), (C, 2), (B, 3), (D, 4)

(c) After the batch

	1	2	3	4
$p=1$: A	1.5	1	0	0
$p=2$: B	0	1	0.5	1
$p=3$: C	1	0.5	1	0
$p=4$: D	0	0	1	1.5

$$s_p = \frac{\sum_k k A[p, k]}{\sum_k A[p, k]}$$

(d) Row means and reorder

$s_1 = 1.4$	(A)
$s_2 = 3.0$	(B)
$s_3 = 2.0$	(C)
$s_4 = 3.6$	(D)

sort by s_p , ties by item ID

$$r_j \leftarrow [A, C, B, D]$$

(e) Reset to identity in the new order

	1	2	3	4
$p=1$: A	1	0	0	0
$p=2$: C	0	1	0	0
$p=3$: B	0	0	1	0
$p=4$: D	0	0	0	1

next batch: $r_j = [A, C, B, D]$

Figure 3: **Accumulation and reorder of one ranking filter.** (a) The accumulator A_j of the filter $r_j = [A, B, C, D]$ starts as the identity in the current order: row p belongs to the item at position p and column k is a position. (b) Two votes from one batch. A vote of weight a toward a target ranking adds $|a|$ to the column of each item's position in the target (3); a negative weight reverses the target first. (c) The accumulator after the batch. (d) The mean voted position s_p of every row (4). Sorting the items by s_p , with ties broken by item ID, gives the new filter $[A, C, B, D]$. (e) The accumulator is reset to the identity in the new order before the next batch.

Table 2: **What a view stores, and what each part is for.** A view is one encoder with the network trained on its rankings, and ArrowFlow trains K of them (Section 5). A target-aware view puts coordinates fitted on the class labels before random ones, and a calibrated view standardizes its projected coordinates (Section 5.1). The output layer serves training and makes none of ArrowFlow’s predictions.

Part	What is fitted or stored	Role
Encoder	imputation and standardization statistics and the projection matrix; a target-aware view also fits linear discriminant coordinates on the training labels, and a calibrated view stores the means and scales of its projected coordinates	turns a real-valued example into a ranking
Hidden ranking layers	one permutation per ranking filter, re-ordered by position votes	produce the ranking that prediction reads
Output layer	one permutation per class	gates the update, returns the signal from which every hidden layer learns, and scores the checkpoint
Nearest-neighbor readout	the last hidden ranking of every training row with its label, and the chosen neighbor count and weighting	predicts the class of one view
Vote across views	the predicted class of each view	gives the prediction of the ensemble

τ , which acts as a push away. The target actually used is therefore $\tilde{\tau} = \tau$ when $a > 0$ and $\tilde{\tau} = \text{rev}(\tau)$ when $a < 0$, and every item of the filter votes for its position in $\tilde{\tau}$:

$$A_j[p, \text{pos}_{\tilde{\tau}}(r_j[p])] \leftarrow A_j[p, \text{pos}_{\tilde{\tau}}(r_j[p])] + |a|, \quad p = 1, \dots, V. \quad (3)$$

A zero weight adds nothing. The vote also returns $\text{sign}(a) m_{r_j \rightarrow \tau}$, the motion of the filter toward the unreversed target, negated when the vote pushes away. The training procedure passes this motion to the layer below.

After all the votes of a batch are in, each row’s mean voted position

$$s_p = \frac{\sum_{k=1}^V k A_j[p, k]}{\sum_{k=1}^V A_j[p, k]}, \quad (4)$$

reorders the filter. The items $r_j[p]$ are sorted by (s_p, ID) , with the row means compared as computed (Table S1). A filter that receives no votes keeps its order. Then A_j is reset to the identity in the new order. So the accumulator keeps no evidence from one batch to the next, and only the updated filter orders carry earlier training forward. Figure 3 traces one update. Ranking items by their mean voted position is a Borda count, a voting rule that scores each item by the positions it receives. Section 4 develops this reading.

3.5 Supervised Training

Training turns classification errors into votes, layer by layer from the top. Each iteration draws a batch and runs the forward pass with the filters held fixed. Then it updates the

layers from the output layer down. In outline, every misclassified example votes to move the filter of its class toward the ranking that the example presents to the output layer, and so does a small random share of the correctly classified ones. The items of the output layer are the filters of the layer below, so the motion returned by this vote has one entry per hidden filter. A positive entry means that the class filter ranks that hidden filter earlier than the example’s ranking does. That hidden filter should come nearer to its input, so it votes toward the input. A filter with a negative entry votes toward the reversed input, which pushes it away. Algorithm 1 gives the complete procedure for one network. Section S2.2 traces it by hand on a four-item example with a tied distance and a gated batch.

Gate. Every example that the prototype readout misclassifies is accepted for the update. A correctly classified example is accepted only with a small probability p_c , and otherwise it sends no signal. The gate acts only in training.

Output layer. For an accepted example of class y and sample weight w , only the class- y filter receives a vote. The vote has weight $w\eta/C$ and points toward the saved input $\pi^{(L-1)}$, and no filter of another class is pushed away. Here the learning rate η scales the votes. Through the vote weights it also scales an equivalent gradient step (Section S1.8). The returned motion is keyed by the hidden filters that the class filter ranks. It becomes the signal list $\mathcal{M}^{(L-1)}$ for layer $L - 1$, with its entries sorted from the largest magnitude down. This motion is exactly half the gradient of the squared position distance between $\pi^{(L-1)}$ and the class filter, taken with respect to the positions of the hidden filters in $\pi^{(L-1)}$ (Section S1.8). Nothing is differentiated to obtain it.

Hidden layers. At layer ℓ with N_ℓ filters, each example selects the first $\lceil \rho N_\ell \rceil$ filters of its signal list, the ones whose signals are largest in magnitude. Here ρ is the selected fraction. A selected filter j whose entry in the signal list is $\mathcal{M}_j^{(\ell)}$ receives the signed vote

$$a_j = \frac{2\eta \mathcal{M}_j^{(\ell)}}{N_\ell} \quad (5)$$

toward the example’s saved input $\pi^{(\ell-1)}$ by (3). If a hidden layer lies below, the layer relays the motions that these votes return, $\text{sign}(a_j) m_{r_j \rightarrow \pi^{(\ell-1)}}$. An attracting vote thus asks the layer below to bring its output closer to filter j , and a repelling vote asks it to move its output away. The relayed motions are averaged, item by item, into u , and the signal passed down is

$$\tilde{u} = \frac{c V_\ell u}{\max_v |u_v| + 10^{-5}}, \quad (6)$$

with c the signal scale. All votes of a layer are cast before any of its filters is reordered by (4). Only some variants of the depth study in Section 7.6 use the relay above. Every other experiment used an earlier version of it, one that did not carry the sign of the vote. After a repelling vote, the earlier relay sent a motion that asked the layer below, to first order, neither to approach the filter nor to move away from it (Section S1.8). The difference affects only networks with two hidden layers (70 of ArrowFlow’s 255 outer folds), because the output layer’s votes only attract.

Algorithm 1 Training one ArrowFlow network and fitting its readout

Require: rankings $\mathcal{D} = \{(\pi, y, w)\}$ of V_1 items with class labels and sample weights; widths $N_1, \dots, N_{L-1}$ and $N_L = C$; iterations T ; batch size B ; initial rate η ; fraction ρ ; scale c ; probability p_c ; validation fraction ν ; schedule (multistep or geometric); output-layer flag (trainable or frozen); readout candidates (neighbor counts and weightings); augmentation switch; seed; the data are nonempty, $L \geq 2$, $C \geq 2$, $V_1 \geq 2$, $T \geq 0$, $\eta > 0$, $c \geq 0$, $w \geq 0$, $0 \leq \rho \leq 1$, $0 \leq p_c \leq 1$, $0 \leq \nu < 1$ with $\mathcal{D}_{\text{tr}}$ nonempty, and T divisible by four under the multistep schedule

- 1: seed the random generator; initialize every filter $r_j^{(\ell)}$ as a uniform random permutation of its vocabulary and every accumulator $A_j^{(\ell)} \leftarrow I$
- 2: $\mathcal{D}_{\text{tr}} \leftarrow \mathcal{D}$; $\mathcal{D}_{\text{val}} \leftarrow \emptyset$; **if** $\nu > 0$ **then** shuffle $\mathcal{D}$; move its first $\lfloor \nu |\mathcal{D}| \rfloor$ rankings to $\mathcal{D}_{\text{val}}$ and keep the rest as $\mathcal{D}_{\text{tr}}$
- 3: **if** augmentation is on **then** extend $\mathcal{D}_{\text{tr}}$ with its perturbed copies (Section 5.3)
- 4: $\Theta^* \leftarrow$ current filters; **if** $\mathcal{D}_{\text{val}}$ is nonempty **then** $\epsilon^* \leftarrow$ prototype-readout error of Θ^* on $\mathcal{D}_{\text{val}}$ ▷ initial checkpoint
- 5: **for** $t = 1, \dots, T$ **do**
- 6: draw $\min(B, |\mathcal{D}_{\text{tr}}|)$ rankings from $\mathcal{D}_{\text{tr}}$ without replacement, all of them if B is unset ▷ batch
- 7: **for all** (π, y, w) in the batch **do** ▷ forward pass with fixed filters
- 8: $\pi^{(0)} \leftarrow \pi$
- 9: **for** $\ell = 1, \dots, L$ **do**
- 10: save $\pi^{(\ell-1)}$ as the input of layer ℓ ; $D_j \leftarrow d_F(\pi^{(\ell-1)}, r_j^{(\ell)})$ for $j = 1, \dots, N_\ell$
- 11: $\pi^{(\ell)} \leftarrow$ filter IDs sorted by (D_j, j)
- 12: **end for**
- 13: $\hat{y} \leftarrow \pi^{(L)}[1]$ ▷ prototype readout
- 14: $g \leftarrow 1$ if $\hat{y} \neq y$; otherwise draw $\xi \sim U(0, 1)$ and set $g \leftarrow 1$ if $\xi < p_c$, $g \leftarrow 0$ if not ▷ gate
- 15: vote $a \leftarrow g w \eta / C$ on $r_y^{(L)}$ toward $\pi^{(L-1)}$ by (3); $\mathcal{M}^{(L-1)} \leftarrow$ its returned motion (zero if $a = 0$)
- 16: save this example's $\mathcal{M}^{(L-1)}$, keyed by the filters of layer $L - 1$, and sort it by $(-|m|, \text{ID})$ ▷ output vote
- 17: **end for**
- 18: **if** the output layer is trainable **then** reorder every $r_j^{(L)}$ by (4); in all cases $A_j^{(L)} \leftarrow I$ for all j
- 19: **for** $\ell = L - 1, \dots, 1$ **do** ▷ hidden layers, top down
- 20: **for all** examples of the batch, with list $\mathcal{M}^{(\ell)}$ and saved input $\pi^{(\ell-1)}$ **do**
- 21: $S \leftarrow$ the first $\lceil \rho N_\ell \rceil$ entries of $\mathcal{M}^{(\ell)}$ ▷ selection
- 22: **for all** $j \in S$ **do** ▷ signed votes; the earlier relay set $d_j \leftarrow m_{r_j^{(\ell)} \rightarrow \text{rev}(\pi^{(\ell-1)})}$ if $a_j < 0$
- 23: $a_j \leftarrow 2\eta \mathcal{M}_j^{(\ell)} / N_\ell$; vote a_j on $r_j^{(\ell)}$ toward $\pi^{(\ell-1)}$ by (3); $d_j \leftarrow \text{sign}(a_j) m_{r_j^{(\ell)} \rightarrow \pi^{(\ell-1)}}$
- 24: **end for**
- 25: **if** $\ell > 1$ **then**
- 26: $u \leftarrow$ mean of $\{d_j : j \in S, a_j \neq 0\}$ aligned by item ID, zero if the set is empty
- 27: $\tilde{u} \leftarrow c V_\ell u / (\max_v |u_v| + 10^{-5})$ ▷ mean signal
- 28: $\mathcal{M}^{(\ell-1)} \leftarrow$ the entries of $\tilde{u}$, keyed by the filters of layer $\ell - 1$ and sorted by $(-|\tilde{u}|, \text{ID})$
- 29: **end if**
- 30: **end for**
- 31: reorder every $r_j^{(\ell)}$ by (4); $A_j^{(\ell)} \leftarrow I$ for all j ▷ reorder and reset
- 32: **end for**
- 33: multistep: $\eta \leftarrow 0.7\eta$ if $t > 2$ and $t \in \{T/4 + 1, T/2 + 1, 3T/4 + 1\}$, with T divisible by four; geometric: $\eta \leftarrow 0.993\eta$ ▷ schedule
- 34: **if** $\mathcal{D}_{\text{val}}$ is nonempty **then** ▷ checkpoint
- 35: $\epsilon \leftarrow$ prototype-readout error of the current filters on $\mathcal{D}_{\text{val}}$; **if** $\epsilon < \epsilon^*$ **then** $\epsilon^* \leftarrow \epsilon$ and $\Theta^* \leftarrow$ current filters
- 36: **end if**
- 37: **end for**
- 38: $\Theta \leftarrow \Theta^*$ if $\mathcal{D}_{\text{val}}$ is nonempty, else the current filters
- 39: $\mathcal{H} \leftarrow$ the last hidden ranking under Θ , with its label, of every ranking in $\mathcal{D}$ ▷ stored hidden rankings
- 40: choose the neighbor count and weighting by cross-validation on $\mathcal{H}$ ▷ readout selection
- 41: **return** Θ , $\mathcal{H}$ and the chosen readout settings

Schedule and checkpoint. The learning rate decays on a fixed schedule (Algorithm 1, line 33). A validation fraction ν of the training rankings is held out before training. After every iteration, the validation error of the prototype readout is measured, and the filters with the lowest such error are the ones returned. Each batch update of a filter exactly minimizes a squared position objective, whose targets change with the network’s predictions (Section S1.8). When the signed vote weights of a batch sum to more than -1 , the update also equals a gradient step followed by a move to the nearest ranking. The step descends a squared position loss of the votes, starting from the filter’s current order. On position vectors this is a step of learning vector quantization, and no derivative is computed to take it (Section S1.8). But no loss of the whole network is minimized. Section 7 measures what the updates add over untrained filters.

3.6 Prediction

The output layer holds one filter per class, which is a single point in the space of permutations. A class may occupy several regions of that space, so ArrowFlow predicts differently. A nearest-neighbor (kNN) readout compares an example’s last hidden ranking with those of all training examples. The output layer then serves only the gate, the returned signal and the checkpoint. Training thus improves one classifier, the nearest class filter, while we evaluate another, the nearest training examples. Proxy-based metric learning makes the same split [Movshovitz-Attias et al., 2017].

After training, the network passes every training ranking through its trained hidden layers. It stores each resulting last hidden ranking $\pi^{(L-1)}$ with its class label. For a new ranking, the readout finds the stored rankings nearest to its last hidden ranking by the footrule, and each of these neighbors votes for its class. A vote has weight one or, under distance weighting, the inverse of the neighbor’s distance. The class with the most weight wins. The neighbor count and the weighting are chosen by cross-validation over the stored training rankings, with the trained layers held fixed (Section S2.1).

4 Arrow’s Impossibility Theorem as a Design Lens

Every update of a ranking filter is an election. The vote targets of a batch are the voters, the items of the layer are the alternatives on the ballot, and the reordered filter is the collective ranking. This section shows what Arrow’s theorem says about ArrowFlow’s update and what it leaves open.

The three axioms. Arrow’s impossibility theorem [Arrow, 1951, 1963] concerns rules that merge the rankings of a fixed finite set of voters into one collective ranking. With three or more alternatives, no rule that is defined on every profile of rankings satisfies all three of the following conditions.

- *Pareto*: if every voter ranks a before b , so does the collective ranking.
- *Independence of irrelevant alternatives* (IIA): the collective order of a and b depends only on how each voter orders a and b . Moving a third alternative around on the ballots cannot change it.

- *Non-dictatorship*: no single voter’s ranking is the collective ranking on every profile.

Each condition sounds harmless on its own. The theorem says that a rule can keep at most two of them.

The update as an election. In one batch update of filter r_j , the voters are the targets $\tilde{\tau}_1, \dots, \tilde{\tau}_n$ of the votes (3) that reach the filter with nonzero weight. Voter t has weight $|a_t|$. The identity prior of A_j adds one more voter, the current order r_j , with weight one. The row-mean reorder (4) therefore ranks the items by the weighted Borda score

$$P_j(v) = \text{pos}_{r_j}(v) + \sum_{t=1}^n |a_t| \text{pos}_{\tilde{\tau}_t}(v), \quad (7)$$

which Proposition 3 derives. An item’s score is its weighted sum of positions, so the item with the smallest score goes first.

Proposition 1 (The update rule and Arrow’s axioms). *Let $V \geq 3$ and fix the voters and their weights so that no voter, the prior included, weighs more than $V - 1$ times the others combined. The weighted Borda count (7) is defined on every profile, satisfies Pareto and is non-dictatorial. By Arrow’s theorem it therefore violates IIA. Section S1 gives the proof and witnesses with and without the prior.*

Each vote in ArrowFlow weighs less than 0.4, so only the prior can break the weight condition. For the prior, the condition says that the votes of the batch weigh at least $1/(V - 1)$ in total. That is exactly the weight at which a batch can first move the filter (Section S1.1).

What follows. The Borda count also shows which context can matter. For two items a and b , (7) gives

$$P_j(b) - P_j(a) = (\text{pos}_{r_j}(b) - \text{pos}_{r_j}(a)) + \sum_{t=1}^n |a_t| (\text{pos}_{\tilde{\tau}_t}(b) - \text{pos}_{\tilde{\tau}_t}(a)).$$

Each voter adds its weight times the number of positions that separate a and b on its ballot, with a minus sign if it puts b first. So the new order of a and b depends only on each voter’s order of the two and on how many items it places between them. Moving a third item into or out of the stretch between them can reverse the pair, and moving it anywhere else cannot. This is the intensity form of independence studied by Saari [1998]. It implies the modified independence that Maskin [2025] uses to characterize the Borda count. Figure 4 shows a small case, in which moving item C between B and A reverses their order.

The proposition fixes the voters and their weights, which in training depend on the predictions. It also treats the prior as a ballot that can change. It concerns every possible profile, and it does not say which profiles arise in training. An actual update holds the current order fixed and maps the votes of a batch to a new filter. In ArrowFlow’s layers the votes of one batch weigh less than $V - 1$ in total, so this map is not Pareto efficient. Arrow’s theorem therefore says nothing about it. The map still violates independence whenever its

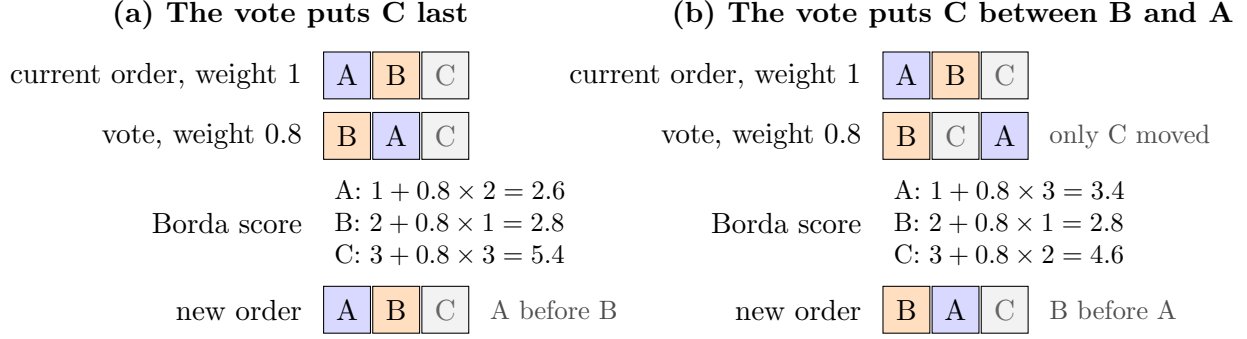


Figure 4: A third item changes the order of two. Two batch updates of one filter over the items A, B and C by the weighted Borda count (7). In each, the filter’s current order votes with weight 1 and one vote of weight 0.8 arrives, which meets the weight condition of Proposition 1. A single vote in ArrowFlow weighs less than 0.4 (Section S1.1), so this vote stands for a few identical votes of one batch. An item’s score is its weighted sum of positions, and the new order sorts the scores from lowest to highest. In both updates the current order puts A before B, and the vote puts B before A. The vote differs only in where it places C. Moving C between B and A raises the vote’s count of items between them from zero to one. That raises A’s score from 2.6 to 3.4 and leaves B’s at 2.8, so the new order of A and B reverses. This is one instance of the violation of independence of irrelevant alternatives that Proposition 1 guarantees. With the current order held fixed, this update lies in the middle range of Figure S1, where the violation follows directly.

votes can move the filter. A direct construction shows this (Section S1.1). So this context dependence lives in the learning rule. The forward pass has none within one layer. Each filter’s distance depends only on the input and that filter, so no third filter can reverse the order of two others. A filter’s position in the output still depends on all the filters of its layer. And Arrow’s theorem predicts nothing about accuracy. The update rule is context dependent whether or not that helps classification.

Three exchangeable stages. The voting reading shows three stages where another aggregation rule could be used. They are the filter update, the readout and the vote across views. Arrow’s theorem concerns only the first, since the other two output a class.

1. The filter update uses the Borda count. One alternative is the footrule median, the permutation with the smallest summed footrule distance to the voters [Dwork et al., 2001].
2. The readout takes a plurality vote among the nearest stored training rankings. Its alternatives are the prototype readout and a readout with several prototypes per class, each built by a Borda count.
3. The K views (Section 5) are combined by majority vote. With the prototype readout, a Borda count over their class rankings can replace it (Section 5.2).

Sections 7.4 and 7.6 and Section S4 report these comparisons. The readout with several prototypes per class was compared on inner folds only.

5 From Real-Valued Data to Permutations: Encoding and Multi-View Ensembles

ArrowFlow ranks items, so real-valued features must first become rankings. Sorting a vector keeps only the order of its coordinates and throws away their sizes. For example, $[1.0, 2.0, 3.0]$ and $[0.01, 100, 100.01]$ give the same ranking. The encoder therefore first creates more coordinates to compare, by polynomial expansion and projection, and only then sorts. Several differently projected views then vote, and the experiments measure whether all this helps prediction.

5.1 Encoding

An encoder maps $x \in \mathbb{R}^d$ to a score vector $z \in \mathbb{R}^e$ and returns $\pi_x = \text{argsort}(z)$, the ranking of the e coordinate IDs by ascending score. It runs the four steps below, and every statistic they use is fitted on the training rows only.

1. *Imputation* replaces a missing value by the training mean of its column.
2. *Polynomial expansion* of degree p replaces x by the vector $\phi(x) \in \mathbb{R}^{d'}$ of all monomials of degree at most p in its coordinates, powers included. These monomials let the sorted scores depend on interactions between features, which changes the set of rankings that the encoder can produce from $\mathbb{R}^d$.
3. *Standardization* centers every column and scales each nonconstant column to unit variance, giving $q(x)$.
4. *Projection* multiplies by a matrix $W \in \mathbb{R}^{d' \times e}$, $z = q(x)W$, drawn by one of three strategies. A *random* projection has independent standard normal entries. A *target-aware* projection places coordinates of a linear discriminant analysis (LDA) before random coordinates. LDA is a linear map fitted on the training rows and their labels to separate the classes. A *calibrated* projection is a random projection whose coordinates are standardized before sorting, so that no coordinate dominates through its variance.

Section S2.3 gives the exact rules of each step. At degree one the encoder also ignores scale. Sliding an example toward the training mean or away from it, along the ray from the mean through the example, leaves all of its rankings unchanged. Only its direction from the mean counts (Section S2.3).

5.2 Multi-View Ensemble

Different projection matrices rank the same input differently, and the ensemble uses this diversity (Figure 1). A *view* is one encoder together with the network trained on its rankings. ArrowFlow trains K views independently, each from its own seed. View k takes the projection strategy at position k of the repeating cycle target-aware, random, calibrated.

Each view predicts with its nearest-neighbor readout (Section 3.6), and a *majority vote* combines the views. The class predicted by the most views wins, even without an absolute majority, and the lowest class ID wins a tie. With the prototype readout, a *Borda count*

over the class rankings $\pi^{(L)}$ can replace the vote, as in classifier combination [Ho et al., 1994, van Erp and Schomaker, 2000].

5.3 Augmentation

Augmentation adds slightly perturbed copies of the training rankings, the permutation analog of small additive noise. Each copy swaps between 1 and $s_{\max}$ random pairs of neighboring items, so it lies within footrule distance $2s_{\max}$ of its original. The originals and the validation rankings are kept unchanged. Whether augmentation is switched on is decided from the training rows alone (Section S2.3).

5.4 Settings

The encoded vocabulary size e , the degree p , the widths, the initial learning rate and the augmentation switch are set from the training rows alone. Some follow a fixed rule, and the others are chosen by an inner cross-validation.

Section 7.1 describes the selection, and Section S2 lists every setting with its symbol and value.

5.5 An Encoder Trained by the Ranking Signal

The encoder above is fixed. Nothing in it changes when the network makes a mistake. This subsection replaces it by an *encoder network*, a small neural network whose output scores are sorted exactly as before. It trains that network with motions of the same kind as those that train ArrowFlow’s hidden layers. This is an extension of ArrowFlow, and it works in two stages. First, each view’s encoder network is trained with its own layer of class filters. Second, the network is fixed and its class filters are set aside. ArrowFlow then trains its ranking layers on the rankings of the trained encoder, as it does with the fixed encoder. The signal that trains the encoder network also has an *away term*, which pushes each coordinate away from the nearest wrong class filter. ArrowFlow’s output vote never does that.

The difficulty is the sort. A neural network learns from gradients, but a ranking has no useful gradient. Nudging a score slightly either leaves the ranking unchanged or makes it jump. The motions offer another route (Figure 5). They say where each coordinate should sit in the ranking, and a position in the ranking corresponds to a score. So each motion can be translated into a target score, and the network can regress toward it.

The encoder network maps x to scores $h \in \mathbb{R}^e$, and the input ranking is $\pi_x = \text{argsort}_{\downarrow}(h)$, the e coordinates in order of decreasing score. The fixed encoder of Section 5.1 sorts in increasing order. Here the order is decreasing, so that position 1 holds the largest score. A ranking layer with one filter per class reads π_x , and its filters keep learning by the output layer’s rule of Section 3.5. For an accepted example, the filter r_y of the true class gives each coordinate a motion $m_y = m_{\pi_x \rightarrow r_y}$ by (1). The filter r_w of the nearest wrong class, the other class whose filter lies nearest to π_x by the footrule, gives it another motion $m_w = m_{\pi_x \rightarrow r_w}$. Coordinate by coordinate, m_y is the motion that the class filter’s vote returns (Section 3.4), seen from the input’s side and so with the opposite sign. Section S6.1 gives the exact gating. The *target conversion* turns these motions into numbers in two steps. First, it picks a target

position. This is the coordinate’s position in the true filter, minus half the displacement that the nearest wrong filter asks of it. Second, it reads the target score off the current scores. The target score is the score that now stands at the resulting position q_p ,

$$\tilde{h}_{\pi_x[p]} = h_{\pi_x[q_p]}, \quad q_p = p + m_y[p] - \frac{1}{2} m_w[p], \quad (8)$$

with q_p kept inside $[1, e]$ and the score interpolated between the two nearest positions. In position space, q_p is one gradient step of unit size on a squared loss. It pulls each coordinate toward its position in the true filter and pushes it half as hard away from its position in the nearest wrong filter. Learning vector quantization moves its prototypes by a similar pull and push [Kohonen, 1990]. The network then takes one step of Adam [Kingma and Ba, 2015], a standard method of gradient-based training, on $\frac{1}{2}\|h - \tilde{h}\|^2$, averaged over the accepted examples of the batch. The target is held fixed during the step. The sort itself is never differentiated. The discrete motion becomes a target in the network’s own units, which is the idea of target propagation [Bengio, 2014, Lee et al., 2015].

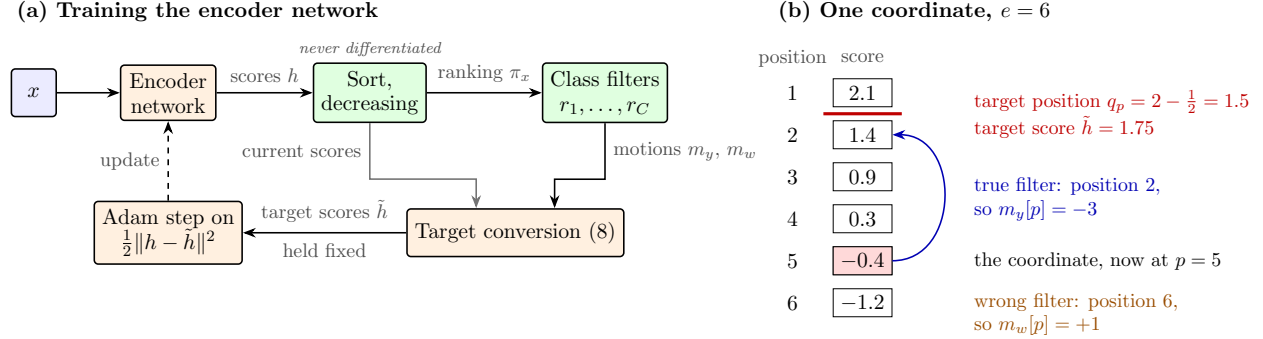


Figure 5: **How the ranking signal trains the encoder network.** (a) The encoder network turns an example x into scores h , and a sort in decreasing order gives the input ranking π_x . A ranking layer with one filter per class reads π_x and keeps learning by its own vote. For an accepted example, the filter of the true class and the filter of the wrong class nearest to π_x give the motions m_y and m_w . The target conversion (8) turns them into target scores $\tilde{h}$. With $\tilde{h}$ held fixed, an Adam step on $\frac{1}{2}\|h - \tilde{h}\|^2$ updates the network (dashed arrow). No gradient passes through the sort. After training, each view’s encoder network is fixed and replaces that view’s fixed encoder. (b) The conversion for one coordinate, with $e = 6$ scores listed from the largest down. The coordinate sits at position $p = 5$ with score -0.4 . The true filter puts it at position 2, so $m_y[p] = -3$ (blue arrow), and the nearest wrong filter puts it at position 6, so $m_w[p] = +1$. Its target position is $q_p = 5 - 3 - \frac{1}{2} = 1.5$ (red line). Its target score, 1.75, lies halfway between the scores at positions 1 and 2, and the loss pulls its score toward that value. The scores are made up for the example.

The encoder network has three fully connected layers of width e , and the first two start with weights drawn uniformly from 0 to 1. It trains for 60 passes over the training rows, with no validation checkpoint. Section 7.7 evaluates the extension, and Section S6.1 gives every setting.

6 Theoretical Analysis

What do the fixed parts of ArrowFlow guarantee? This section answers four questions.

1. When does the encoded ranking stay the same?
2. Which rankings can the encoder produce?
3. How far can an input move before a fixed layer changes its output?
4. What does one batch update compute?

Each statement concerns a fixed encoder, layer or batch. None of them is a convergence or generalization theorem for the trained network. Section S1 states and proves every result that we summarize here in words.

6.1 Stability of the Ordinal Encoding

The encoded ranking $\pi_x = \text{argsort}(z)$ changes only when two coordinates of the score vector $z \in \mathbb{R}^e$ (Section 5) cross. So the smallest gap between two scores, $\delta_{\min}(z) = \min_{i < j} |z_i - z_j|$, controls how stable the ranking is.

Proposition 2 (Score-gap stability). *Let $z \in \mathbb{R}^e$, $e \geq 2$, have distinct coordinates, and let $\varepsilon \in \mathbb{R}^e$ satisfy $\|\varepsilon\|_\infty < \delta_{\min}(z)/2$. Then $\text{argsort}(z + \varepsilon) = \text{argsort}(z)$.*

In words, if every score moves by less than half the smallest gap, the ranking stays the same. The condition is sufficient but not necessary. Its bound cannot be raised, because the two closest scores cross when each moves toward the other by more than half their gap (Figure 6b). Nor does it give a noise threshold, because Gaussian noise exceeds any bound with positive probability. For Gaussian input noise and an encoder of degree one, Section S1.3 bounds the probability that the ranking changes.

6.2 The Ordinal Alphabet

Ranking keeps order and discards magnitude, so the encoder can output at most $e!$ rankings. All score vectors without ties that share one ranking π form an open convex cone, $\{z : z_{\pi[1]} < z_{\pi[2]} < \dots < z_{\pi[e]}\}$, cut out by the hyperplanes $z_i = z_j$ (Figure 6). A vector with tied scores lies on a wall of a cone and gets the ranking that the ID rule assigns. The encoding is constant inside a cone and jumps at its walls. A fixed projection may land in only some of these cones. So $e!$ bounds the alphabet of rankings that the encoder can emit, but it does not measure the information the data carry (Section S1.5).

At degree one the alphabet can be counted. Every score is then the dot product of the d standardized features with a fixed vector of the view, a column of W for a random view. So the ranking orders e fixed points in d dimensions by their projection onto one direction. Cover [1967] counted how many orderings such projections can produce. A view of degree one reaches at most that many rankings, and a random view can reach all of them. For $e = 16$ and $d = 4$ there are 437,040, against $16! \approx 2.1 \times 10^{13}$ (Section S1.5). Polynomial expansion makes the scores linear in more coordinates, so it can enlarge this set.

For the random projection, inputs that point in similar directions get similar rankings. Suppose that the expanded, standardized features of two fixed inputs form the angle θ .

Then, in expectation over the random projection matrix, their rankings disagree on $\binom{e}{2}\theta/\pi$ coordinate pairs (Section S1.6).

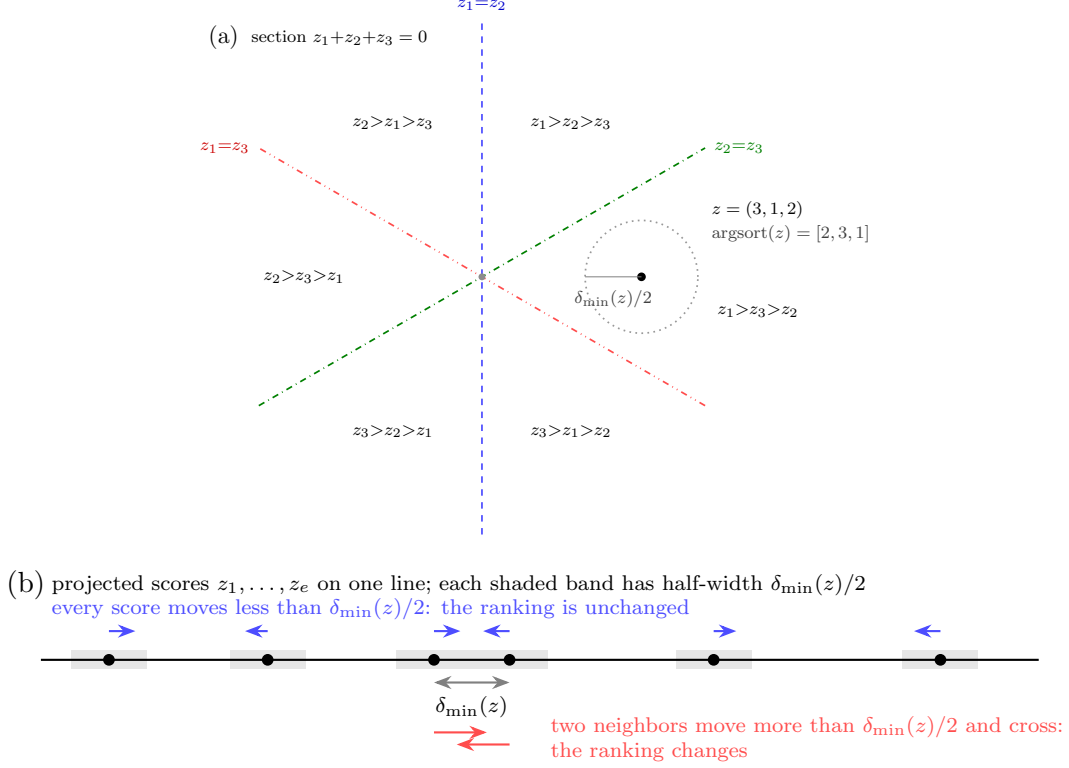


Figure 6: **Permutation cones and the stability radius.** (a) Up to the hyperplanes $\{z_i = z_j\}$, whose points the tie rule assigns, the argsort encoding partitions the projected space $\mathbb{R}^e$ into $e!$ open convex cones (Weyl chambers). They are drawn here for $e = 3$ in the plane $z_1 + z_2 + z_3 = 0$, which every cone meets and which the three hyperplanes cut into six sectors. All points in a cone map to the same permutation. The point $z = (3, 1, 2)$ is drawn at its position in the plane, which differs from z by a multiple of the all-ones vector and has the same ranking and the same coordinate gaps. The dotted circle has radius $\delta_{\min}(z)/2$, half the smallest coordinate gap of Proposition 2. It lies inside the region $\|\epsilon\|_\infty < \delta_{\min}(z)/2$, so a perturbation inside it cannot cross a boundary. The nearest wall is farther away, at distance $\delta_{\min}(z)/\sqrt{2}$. (b) The same condition on one line of projected scores. Scores that each move by less than $\delta_{\min}(z)/2$ keep their ranking, and two neighbors that move further can cross.

6.3 Margins and Ties of a Fixed Ranking Layer

A fixed layer is stable in the same way. The footrule is a metric, so no filter’s distance changes by more than the input moves. When all distances differ, the smallest gap between consecutive distances therefore guarantees the same output for every input within half that gap (Section S1.7). The guarantee fails where distances tie. Footrule distances are even integers up to $\lfloor V^2/2 \rfloor$ [Diaconis and Graham, 1977], so at most $\lfloor V^2/4 \rfloor + 1$ values occur. A layer with more filters than that has tied distances at every input. For example, a layer of

128 filters over $V = 16$ items can see at most 65 distinct distances. Ties are common well before that bound. The distances from an input to random filters crowd around their mean $(V^2 - 1)/3$ with a standard deviation near $0.21 V^{3/2}$. At the hidden-layer sizes used here, random filters therefore leave almost no input with all distances distinct, and the guarantee then covers at most the nearest filter (Section S1.7). Where distances tie, the tie rule of Section 3.3 decides the output order.

6.4 What One Batch Update Computes

One batch update is a weighted Borda count of the accepted votes and the prior. In general it is not the footrule median. Because the accumulator is reset after every batch, it keeps no running estimate of where each item belongs on average. Each update depends only on the filter’s current order and the current batch. With little vote weight, the current order can even hold against the average vote (Section S1.9).

Proposition 3 (One batch update is a weighted Borda count). *Fix a filter r_j and the targets $\tilde{\tau}_1, \dots, \tilde{\tau}_n$ of one batch with vote weights $|a_1|, \dots, |a_n| > 0$. After accumulation, the mean voted position s_p of (4) for the row $p = \text{pos}_{r_j}(v)$ of item v equals*

$$s_p = \frac{\text{pos}_{r_j}(v) + \sum_{t=1}^n |a_t| \text{pos}_{\tilde{\tau}_t}(v)}{1 + \sum_{t=1}^n |a_t|} = \frac{P_j(v)}{1 + \sum_{t=1}^n |a_t|}, \quad (9)$$

so in exact arithmetic the reordered filter is the ranking by P_j of (7) with the fixed tie rule. It differs in general from the footrule median of the same profile, even when the mean positions are distinct.

The weighted footrule median minimizes the summed absolute position differences to the voters, and minimum-cost matching computes it exactly [Dwork et al., 2001]. The Borda ranking instead minimizes the summed squared differences [Zwicker, 2008]. For equal-weight votes, it is also the maximum-likelihood center estimate in a ranking model based on this squared distance [Fligner and Verducci, 1986] (Section S1.8). The profiles of Section S1.8 show that the two rules do disagree, on a finite profile and in the limit. Picture an idealized estimator that kept every vote drawn from one fixed distribution. It would converge to the ranking by expected position when the expected positions differ. The reset update need not converge to it, and in the network the votes also change with the current predictions (Section S1.9).

7 Results

The experiments ask six questions.

1. How accurate is ArrowFlow next to standard classifiers tuned in the same way?
2. Does training the ranking filters help, compared with leaving them in their random starting orders?
3. What does each part of the method contribute?

4. Is it the particular signal sent down from the output layer that helps, or would any movement of the filters do?
5. What does training change inside the network, and where does its benefit stop?
6. Can motions of the same kind train a real-valued encoder network, and does ArrowFlow gain from it?

In brief, ArrowFlow trails the best tuned classical model on all seventeen datasets, and training adds only a modest gain. The signal that training sends down does carry information. Yet a fixed kernel on rankings from the same encoder family is more accurate on most datasets. So the paper’s contribution is the learning rule itself. Motions of the same kind also train a real-valued encoder network. With the trained encoder, ArrowFlow has the higher mean accuracy on 11 of the 17 datasets, significantly on one.

7.1 How We Tested

A fair test needs two things. It needs datasets that did not shape the method, and it needs a way of choosing settings that never looks at the test data. Section S2.5 gives the complete protocol. The author used generative artificial intelligence tools for software development, running and analyzing the experiments, and editing the manuscript, and reviewed all of their output. The tools were Claude Opus 5, Claude Opus 5.5, Claude Fable 5 and Claude Fable 5.1 (Anthropic), and GPT 5.6 Sol and GPT 6 Astra (OpenAI).

Datasets. We use seventeen classification datasets. Seven are standard benchmarks from the University of California, Irvine (UCI) Machine Learning Repository [Dua and Graff, 2019], and they served during the method’s development. They are Iris [Fisher, 1936], Wine [Aeberhard and Forina, 1992], Breast cancer [Wolberg et al., 1993], Wine quality (red wines, three quality classes) [Cortez et al., 2009], Vehicle [Mowforth and Shepherd, n.d.], Segment [Image Segmentation, 1990] and Digits [Alpaydin and Kaynak, 1998]. Wine quality’s three classes are the quality scores 5 or less, 6, and 7 or more. The other ten, the further datasets, were fixed in advance as every dataset of a predefined pool of OpenML datasets [Vanschoren et al., 2014, Bischl et al., 2021] that met prespecified eligibility rules, as interpreted during selection and before any model was fitted (Section S3.1). They are balance-scale [Siegler, 1976], mfeat-zernike [Duin, 1998], ionosphere [Sigillito et al., 1989], vertebra-column [Barreto and Neto, 2005], diabetes [Smith et al., 1988], banknote-authentication [Lohweg, 2012], qsar-biodeg [Mansouri et al., 2013], steel-plates-fault [Buscema et al., 2010], climate-model-simulation-crashes [Lucas et al., 2013] and the hepatitis C virus (HCV) data [Kamal et al., 2017]. The method, its candidate settings and the comparison models were fixed before these ten were selected, and none of the ten had been used in the method’s earlier development (Section S5). The encoder network of Section 5.5 is an exception. Its target conversion and settings were developed on single splits of twelve of the seventeen datasets, six of them further datasets (Section S6.2). All seventeen are small: 150 to 2,310 rows, 4 to 64 features and 2 to 10 classes (Section S5.2).

How settings are chosen. Every tuned setting is chosen by nested cross-validation, so within a run the test data never influence a choice. Each dataset is split into five parts that

keep its class proportions. Each part serves once as the test set, and the split is repeated three times with different shuffles. So every model is tested on the same 15 test sets, called outer folds. All tuning happens inside the other four parts of each split, its training partition [Cawley and Talbot, 2010]. These are split again into three inner folds, and each model picks its configuration there from at most 24 candidates. A stochastic model, one whose fit depends on a random seed, has its three best candidates rescored with all three fitting seeds. The chosen configuration is refitted on the whole training partition and scored once on the test set.

Models. ArrowFlow always uses $K = 7$ views, the nearest-neighbor readout and the validation checkpoint. Its 16 candidates vary the hidden-layer widths, the learning rate and two encoder settings (Section S2.4). The comparison models are five scikit-learn classifiers [Pedregosa et al., 2011], each tuned in the same way. They are a support vector classifier (SVC) with a radial basis function kernel, a random forest, a multilayer perceptron (MLP), numeric kNN on the standardized raw features and gradient boosting. The majority class gives a baseline. ArrowFlow also chooses its readout’s neighbor count and weighting inside every fit, outside the candidate budget.

How differences are judged. Errors are in percent and differences in percentage points (pp). Tables give each model’s mean error over the 15 test sets, with its standard deviation (SD) across them. To compare two models, we take their accuracy difference on each test set and report the mean with a 95% interval. The 15 splits share training data, so their differences are correlated. We use the corrected resampled t interval, which is wider to allow for that [Nadeau and Bengio, 2003, Bouckaert and Frank, 2004]. Over many datasets some differences look large by chance alone. We therefore correct the p values for the number of comparisons with the Holm method [Holm, 1979], and a difference is significant when its corrected p value is below 0.05. Each correction covers a group of comparisons that a run protocol fixed before they were scored. Each such protocol was frozen before its run, with its hash and time recorded, but none was deposited with an external registry. Analyses defined afterwards are exceptions, and we call them post hoc. On the seven benchmark datasets, the training comparisons were fixed after ArrowFlow’s own results there were known (Section S2.5).

7.2 How Accurate Is ArrowFlow?

How does ArrowFlow compare with standard classifiers that get the same data, the same folds and the same cap on candidates? Table 3 gives the test error of every model on every dataset.

ArrowFlow is never the most accurate model. On every one of the seventeen datasets the best tuned classical model has lower error, by 0.19 (banknote-authentication) to 5.32 percentage points (ionosphere). Ranked within each dataset and averaged, ArrowFlow comes fourth of the six tuned models, between random forest and gradient boosting. On the ten further datasets alone it comes fifth, ahead of numeric kNN only (Section S3.2). Accuracy can hide a weakness on a rare class. Balanced accuracy averages, over the classes, the share of

each class’s rows that a model gets right. On climate-model-simulation-crashes, ArrowFlow’s balanced accuracy is 51.2 percent, barely above the 50 percent of the majority class. The support vector classifier reaches 76.1 percent (Section S3.2).

7.3 Does Training Help?

Random ranking filters already turn an input into a new ranking, and the nearest-neighbor readout may work well on it without any training. To see what the learning rule adds, we compare ArrowFlow with two controls that train no filters but are otherwise tuned exactly like it (Table 4). The untrained ArrowFlow keeps its random initial filters. Tuned input footrule kNN skips the ranking layers and applies the nearest-neighbor readout directly to the encoded input rankings.

Training helps, but modestly. ArrowFlow is more accurate than the untrained ArrowFlow on 16 of the 17 datasets. A post hoc one-sided signed-rank test supports the gain over all seventeen ($p < 0.001$) and over the ten further datasets alone ($p = 0.007$; Section S3.3). This test ranks the differences across datasets by size and asks whether the positive ones outweigh the negative ones more than chance allows [Demšar, 2006]. Yet the untrained version comes within one percentage point on 12 of the 17. The gain is significant against at least one control on three datasets: Vehicle, against tuned input footrule kNN, and balance-scale and mfeat-zernike, against both controls. Only the last two survive a stricter, post hoc correction over all 34 comparisons (Section S3.3). Table 4 also checks whether sorting itself helps. On the seven benchmark datasets, sorting the projected scores made no detectable difference to a nearest-neighbor readout (Section S3.4). On the ten further datasets, where the comparison is descriptive, sorting raised the readout’s error on six of them, by up to 2.5 points.

One prespecified test asks where training helps most. Before the runs, four further datasets, balance-scale, mfeat-zernike, ionosphere and vertebra-column, were marked because the published results we counted [Vanschoren et al., 2014, Fernández-Delgado et al., 2014] put tuned kNN at least 5 percentage points behind the best tuned model (Section S3.1). Training gained more on these four than on the other six ($p = 0.033$). But by construction these datasets also leave nearest neighbors more room to improve. So the test cannot tell whether training helps where nearest neighbors suit the raw features poorly, or simply where there is more to gain. It also rests on only ten datasets, and without the three datasets that Table 3 flags it gives $p = 0.20$, a post hoc check (Section S3.3).

7.4 What Does Each Part Contribute?

ArrowFlow combines several parts: the nearest-neighbor readout, seven views, a validation checkpoint and augmentation. To see what each contributes, we remove or replace one part at a time and keep everything else, including the configuration ArrowFlow chose on each split.

The readout makes the largest difference. Predicting with the prototype readout, the nearest output filter, instead of the nearest stored training examples is less accurate on 16 of the 17 datasets, by more than one percentage point on eight of them. Seven views are more accurate than the first, target-aware view alone on 12 of the 17. Removing the checkpoint

Table 3: **ArrowFlow against five tuned classical models and the majority class on the seventeen datasets.** Mean test error in percent over the 15 outer folds (5 folds $\times$ 3 repeats), with the standard deviation (SD) across folds in parentheses; the SD describes spread, not a confidence interval. Every model chose its configuration on inner folds only (Section 7.1). Bold marks the lowest unrounded error among ArrowFlow and the five classical models. The last column gives ArrowFlow’s error minus that of the best tuned classical model, the majority class excluded, in percentage points, positive when ArrowFlow trails; it is positive on all seventeen datasets. It is computed from unrounded errors, so it can differ in the last digit from the printed errors; Section S3.2 lists the errors of the tuned models to two decimals. The marks are post hoc flags. c: the best tuned classical model has less than 1% error. m: ArrowFlow is within 0.5 points of the majority class, and the best tuned model is at least 2 points better. n: no model has lower error than the majority class. SVC: support vector classifier with a radial basis function (RBF) kernel; MLP: multilayer perceptron; kNN: nearest-neighbor classifier; HCV: hepatitis C virus.

Dataset	ArrowFlow	SVC (RBF)	Random forest	MLP	Numeric kNN	Gradient boosting	Majority class	Gap to best tuned (pp)
<i>Benchmark datasets</i>								
Iris	5.0 (3.5)	4.0 (3.4)	5.3 (4.3)	4.1 (3.7)	4.7 (3.3)	5.6 (3.5)	66.7 (0.0)	+1.04
Wine	3.2 (2.9)	1.5 (2.3)	1.8 (1.6)	1.9 (2.2)	3.4 (3.4)	2.6 (2.5)	60.1 (1.6)	+1.69
Breast cancer	2.7 (1.1)	2.3 (1.5)	4.3 (1.5)	2.9 (1.5)	3.5 (1.5)	3.9 (1.6)	37.3 (0.4)	+0.33
Wine quality	27.7 (2.1)	33.1 (3.4)	27.2 (2.3)	32.3 (2.6)	28.8 (2.2)	30.7 (2.1)	53.5 (0.1)	+0.47
Vehicle	19.9 (1.5)	16.5 (2.0)	25.7 (2.3)	17.2 (2.0)	29.3 (2.6)	24.3 (2.2)	74.2 (0.3)	+3.40
Segment	2.6 (0.7)	3.3 (0.9)	1.9 (0.7)	2.8 (0.9)	3.4 (1.1)	2.1 (0.6)	85.7 (0.0)	+0.66
Digits	2.8 (0.9)	1.8 (0.6)	2.4 (0.7)	2.3 (0.8)	2.1 (0.7)	3.3 (0.9)	89.9 (0.2)	+0.97
<i>Further datasets</i>								
balance-scale	6.0 (1.3)	2.1 (1.8)	13.8 (2.1)	2.2 (1.1)	10.2 (1.5)	8.4 (0.8)	54.2 (0.3)	+3.95
lfeat-zernike	19.3 (1.0)	17.1 (1.6)	22.3 (2.0)	18.9 (1.2)	19.6 (1.7)	22.4 (1.8)	90.0 (0.0)	+2.18
ionosphere	11.2 (3.4)	5.9 (2.7)	7.0 (2.5)	8.0 (2.6)	10.2 (3.5)	6.6 (2.1)	35.9 (0.4)	+5.32
vertebra-column	16.9 (3.6)	14.0 (5.2)	15.8 (3.2)	15.4 (4.4)	19.8 (4.0)	17.8 (4.3)	51.6 (0.0)	+2.90
diabetes	24.0 (2.5)	22.7 (2.9)	23.9 (2.6)	23.5 (1.5)	24.5 (2.7)	24.0 (3.2)	34.9 (0.2)	+1.34
banknote-authentication ^c	0.2 (0.5)	0.0 (0.1)	0.9 (0.7)	0.0 (0.0)	0.3 (0.5)	0.3 (0.3)	44.5 (0.1)	+0.19
qsar-biodeg	12.6 (1.8)	12.2 (3.1)	13.0 (2.1)	12.0 (2.9)	13.7 (2.7)	13.2 (2.5)	33.7 (0.2)	+0.52
steel-plates-fault	24.8 (1.5)	23.9 (2.1)	21.2 (1.6)	24.2 (1.7)	25.5 (1.4)	21.0 (1.8)	65.3 (0.1)	+3.82
climate-model-simulation-crashes ^m	8.3 (0.6)	5.4 (2.2)	6.4 (1.1)	5.6 (1.7)	6.8 (1.1)	5.9 (1.4)	8.5 (0.4)	+2.96
HCV ⁿ	75.6 (1.7)	75.5 (2.0)	75.9 (1.6)	75.0 (1.8)	74.1 (2.6)	76.9 (2.4)	73.9 (0.2)	+1.50

Table 4: **What training adds, on the seventeen datasets.** Upper part: mean test error in percent, lower is better, of five classifiers that all predict from nearest neighbors. From left to right they are numeric kNN on the standardized raw features, numeric kNN on the projected scores before sorting, tuned input footrule kNN on the encoded input rankings, the untrained ArrowFlow and ArrowFlow. Neighboring columns differ in more than one component, so this ladder describes and does not decompose. Lower part: ArrowFlow’s accuracy minus that of each control in percentage points, positive when ArrowFlow is more accurate, with its 95% interval, which is not corrected for multiple comparisons. Holm p is corrected over the comparisons named in the block header, which the run protocols fixed before the controls were scored; the benchmark family was fixed after ArrowFlow’s own benchmark results were known. ‡ : still below 0.05 after one post hoc correction over all 34 comparisons. Both controls share ArrowFlow’s encoder family but choose their own settings on inner folds. kNN: nearest-neighbor classifier; HCV: hepatitis C virus.

Dataset	Numeric kNN, raw features	Numeric kNN, projected scores	Tuned input footrule kNN	Untrained ArrowFlow	ArrowFlow
<i>Benchmark datasets</i>					
Iris	4.7	4.0	5.6	6.0	5.0
Wine	3.4	3.9	4.9	3.9	3.2
Breast cancer	3.5	3.3	3.6	3.2	2.7
Wine quality	28.8	28.2	28.1	27.8	27.7
Vehicle	29.3	25.1	24.7	23.4	19.9
Segment	3.4	3.6	3.8	3.6	2.6
Digits	2.1	2.4	2.8	3.0	2.8
<i>Further datasets</i>					
balance-scale	10.2	10.3	10.9	11.0	6.0
mfeat-zernike	19.6	19.0	21.5	21.6	19.3
ionosphere	10.2	12.3	11.2	11.4	11.2
vertebra-column	19.8	16.7	17.8	18.0	16.9
diabetes	24.5	24.3	24.0	24.8	24.0
banknote-authentication	0.3	0.2	0.1	0.2	0.2
qsar-biodeg	13.7	13.8	14.0	13.5	12.6
steel-plates-fault	25.5	25.4	25.2	25.6	24.8
climate-model-simulation-crashes	6.8	7.6	8.3	8.4	8.3
HCV	74.1	75.6	75.7	75.2	75.6
Dataset	ArrowFlow – untrained (pp)	Holm p	ArrowFlow – tuned input footrule kNN (pp)	Holm p	
<i>Benchmark datasets: Holm correction over fourteen comparisons</i>					
Iris	+0.96 [−1.35, +3.28]	1.000	+0.52 [−2.25, +3.29]	1.000	
Wine	+0.74 [−1.13, +2.61]	1.000	+1.68 [−1.04, +4.40]	1.000	
Breast cancer	+0.55 [−0.44, +1.53]	1.000	+0.88 [−0.10, +1.86]	0.751	
Wine quality	+0.11 [−1.44, +1.66]	1.000	+0.44 [−1.23, +2.10]	1.000	
Vehicle	+3.51 [+1.27, +5.74]	0.060	+4.81 [+1.86, +7.75]	0.049	
Segment	+1.06 [+0.24, +1.88]	0.161	+1.18 [+0.39, +1.97]	0.075	
Digits	+0.22 [−0.70, +1.13]	1.000	−0.03 [−1.01, +0.96]	1.000	
<i>Further datasets: Holm correction over ten comparisons per control</i>					
balance-scale	+4.98 [+3.38, +6.57]	<0.001 [‡]	+4.91 [+2.98, +6.83]	<0.001 [‡]	
mfeat-zernike	+2.34 [+1.28, +3.40]	0.003 [‡]	+2.18 [+1.09, +3.28]	0.007 [‡]	
ionosphere	+0.16 [−3.60, +3.92]	1.000	−0.00 [−2.97, +2.97]	1.000	
vertebra-column	+1.08 [−2.43, +4.58]	1.000	+0.97 [−1.52, +3.46]	1.000	
diabetes	+0.71 [−1.59, +3.01]	1.000	−0.03 [−1.52, +1.47]	1.000	
banknote-authentication	+0.03 [−0.25, +0.31]	1.000	−0.09 [−0.68, +0.50]	1.000	
qsar-biodeg	+0.92 [−0.14, +1.97]	0.663	+1.42 [+0.45, +2.39]	0.056	
steel-plates-fault	+0.72 [−1.23, +2.67]	1.000	+0.39 [−1.31, +2.08]	1.000	
climate-model-simulation-crashes	+0.02 [−0.43, +0.48]	1.000	−0.04 [−0.65, +0.57]	1.000	
HCV	−0.37 [−2.59, +1.86]	1.000	+0.13 [−2.73, +2.99]	1.000	

or augmentation changes mean accuracy by less than 0.2 percentage points. These configurations were chosen for ArrowFlow’s own readout, so the comparison with the prototype readout favors ArrowFlow (Section S4.1).

7.5 Is It the Signal That Helps?

Training might help simply because it moves the filters, whatever the direction. To test whether the particular signal is what helps, we keep everything else identical and tamper only with the motions that the output layer sends down to the hidden filters. We compare ArrowFlow with three controls.

- With *frozen filters*, the hidden filters stay in their random starting orders, and everything else is ArrowFlow’s own, settings included (Section S3.6).
- With *permuted alignment*, each motion is delivered to a filter of the same layer drawn at random, afresh in every batch.
- With *random directions*, the directions are shuffled among the motions, so a filter may be pushed away from an input it should approach.

The two scrambled controls keep the number of motions, their sizes and their total weight.

A scrambled signal leaves the network less accurate than no training at all. Delivering the motions to the wrong filters does so on 16 of the 17 datasets, and shuffling their directions on 14 of the 17. These counts are descriptive, but a post hoc one-sided signed-rank test across the datasets supports both ($p < 0.001$; Table S28). The neighborhoods that the readout uses degrade in the same way. We measure them by their purity, how often an example’s nearest training examples share its class. On most datasets, an example’s nearest training examples share its class less often after either scramble than with frozen filters (Figure 7b and Table S29). So the motions carry information about which filter should move and in which direction. A permutation fixed for the whole of training, the analog of feedback alignment [Lillicrap et al., 2016], was not tested (Section S3.6).

Figure 7a compares ArrowFlow itself with each control. Against frozen filters ArrowFlow is more accurate on 14 of the 17 datasets, fewer than the 16 against the untrained ArrowFlow of Section 7.3, which chooses its own settings. The three exceptions lie within 0.2 points of zero, and the mean gain is about one point against either. The two scrambles differ in strength, so each is compared with frozen filters rather than with the other (Section S3.6).

7.6 What Training Does and Where It Stops

The studies below look inside the trained networks and test the limits of the approach.

What does training teach the network? One dataset makes this easy to see. In the balance-scale data, each example places a weight at some distance on each side of a balance. The task is to say whether the balance tips left, tips right or stays level. A level balance is rare, about one example in thirteen, and it is hard to spot, because it sits exactly on the line between the other two outcomes. Before training, ArrowFlow almost never recognizes a level balance, and neither do simpler nearest-neighbor methods on the same inputs. None of them gets more than 5 in 100 right. After training, ArrowFlow gets about half of them

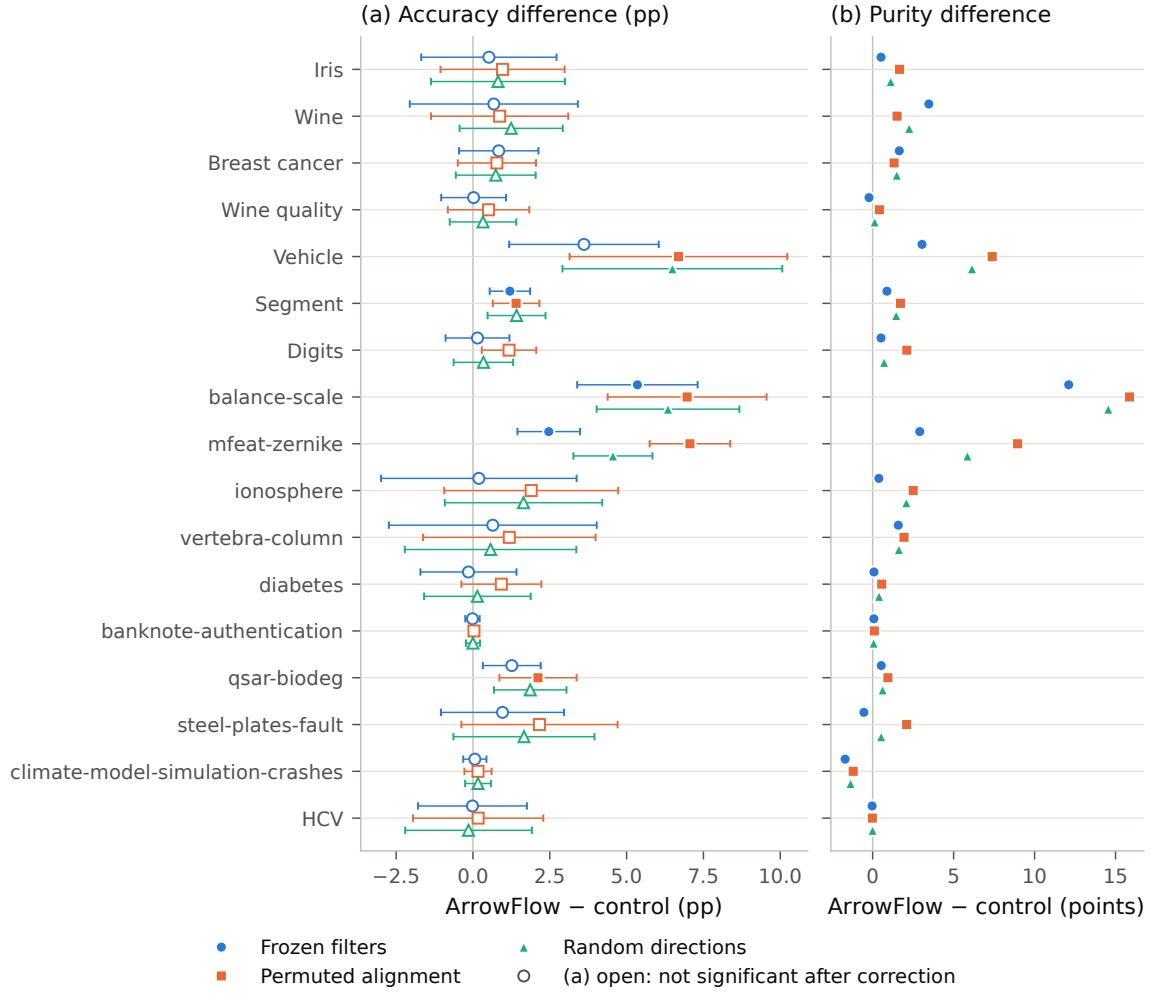


Figure 7: **Is it the signal that helps?** (a) ArrowFlow’s accuracy minus that of each control, in percentage points, with 95% intervals (not simultaneous); positive favors ArrowFlow. Frozen filters: the hidden filters never move. Permuted alignment: each motion reaches a filter of the same layer drawn at random, afresh in every batch. Random directions: the directions of the motions are shuffled. Both scrambled controls keep the number of motions, their sizes and their total weight. Filled marks in panel (a) remain significant after a Holm correction across the seventeen datasets within each control: 3 datasets for frozen filters, 5 for permuted alignment and 3 for random directions. The intervals are not adjusted for multiple comparisons, so an open mark’s interval can exclude zero. In panel (a), ArrowFlow is more accurate than frozen filters on 14 of the 17 datasets, than permuted alignment on 17 and than random directions on 16. (b) Neighborhood purity, how often an example’s nearest training examples share its class: ArrowFlow minus each control, in percentage points, on the test rows; descriptive, with no interval and no test, so its marks carry no fill coding. ArrowFlow is purer than all three controls on 13 of the 17 datasets. In both panels, a scramble’s mark to the right of the mark of frozen filters means that the scramble is less accurate than frozen filters, or in panel (b) less pure. Tables S28 and S29 give these comparisons directly. HCV: hepatitis C virus.

right. The two common outcomes are easy either way. ArrowFlow recognizes tips to either side nearly always, before training and after. So most of what training adds on this dataset comes from learning this one thin class. The support vector classifier recognizes nearly every level balance, far more often than ArrowFlow, so other methods can find this class too (Section S3.7).

Do the filters move at all? The validation checkpoint could, in principle, hand back the random starting filters. It never did. In none of the 1,785 trained views of one fitting seed did the checkpoint keep the initial filters, and in the networks with one hidden layer nearly every hidden filter ended in a new order. Ties limit what the layers can express, though. The distances from an input to many filters bunch together in a narrow band (Section 6.3). So two filters often sit at exactly the same distance from an input, and their IDs then decide their order. In a hidden layer of 128 filters that ranks 32 items, about four in five distances are shared with another filter, before training and after, as with random filters (Section S3.7).

Does a second hidden layer help? It did not. We refitted every split with one hidden layer of 128 filters and with two, of 64 and then 128 filters. Every other setting, including the 200 updates and the learning rate, was the one chosen for that split. On 185 of the 255 splits, that choice was made for a network with one hidden layer. With the earlier relay, one hidden layer was more accurate than two on 15 of the 17 datasets, although no difference was significant after correction. With the corrected relay, two layers were about as accurate as one, and a second layer still did not help (Section S3.7). Over the datasets, two layers minus one averaged -0.24 points, with a post hoc 95% interval from -0.60 to $+0.12$.

Does the voting rule make a difference? Each update merges its votes by a weighted Borda count, one of the three exchangeable stages of Section 4. We replaced it in the hidden layers by the footrule median, the ranking with the smallest total footrule distance to the votes. Its prior weight, the weight of the filter’s current order, was set once, on training data from two datasets, so that it moved the filters about as often as the Borda count there. Over all datasets it moves them less often, and per dataset the movement differs widely (Section S3.7). The Borda count was more accurate on 13 of the 17 datasets, but significantly so only on Vehicle. This compares two voting rules and does not test Arrow’s theorem (Section S3.7).

Do established methods do as well? Could an established method match ArrowFlow without any learned filters? On most datasets, yes. Take a support vector classifier with a fixed Kendall kernel [Jiao and Vert, 2015, 2018]. It compares two rankings by Kendall’s τ , the share of item pairs that they order alike minus the share that they order differently. Applied to rankings from ArrowFlow’s own encoder, with its encoder settings chosen on its own inner folds, the classifier is more accurate than ArrowFlow on 14 of the 17 datasets. The angle property of Section 6.2 suggests why. For a random view, the expected τ of two encoded rankings over the draw of the projection is $1 - 2\theta/\pi$, where θ is the angle between their expanded, standardized features (Section S1.6). Up to a scale and a shift, this is the arc-cosine kernel of order zero [Cho and Saul, 2009], so the classifier acts much like a kernel

machine on the features. ArrowFlow has a higher mean accuracy than nearest neighbors after a linear discriminant projection on 12 of the 17 datasets, and after a principal component projection on 11. After a learned metric (neighborhood components analysis), it has the higher mean accuracy on only 7 of the 17 (Section S3.7).

Does training improve the representation itself? ArrowFlow reads its hidden rankings with a nearest-neighbor vote. Training could improve the rankings themselves, or it could only arrange them to suit that vote. To tell the two apart, we read the untrained and the trained hidden rankings with a strong fixed readout, a support vector classifier with the same Kendall kernel. On the ten further datasets, the scope fixed in advance, the trained rankings had the higher mean accuracy on only 3, and no difference was significant. Over the ten datasets, trained minus untrained averaged -0.10 points, with a post hoc 95% interval from -0.56 to $+0.35$. For this outcome, the protocol had fixed the following statement before the test. Training improves the nearest-neighbor neighborhoods but not the representation read by a strong fixed readout (Section S3.7).

7.7 Can the Ranking Signal Train the Encoder?

The motions have so far trained ranking filters. Here we ask whether they can also train the real-valued weights of the encoder network of Section 5.5, and whether ArrowFlow gains from the trained encoder. The answer to the first is yes, and the answer to the second is a modest yes, by a rule fixed before the run (Figure 8). Both studies below use the benchmark's test sets, but the target conversion and its settings were developed on single splits of twelve of its seventeen datasets (Section 7.1).

Does the encoder network learn? We tested the encoder network with one ranking layer of class filters on top, tuned on the same inner folds and scored on the same test sets (Section S6.3). Trained by the converted motions, it was more accurate than the same network left at its random start on all 17 datasets, significantly so on 11 (Figure 8a). So a discrete ranking signal trains continuous weights without differentiating the sort, as other methods that keep the ranking step exact also do (Section 2). The untrained encoder is a weak reference, though. Its first two layers start with weights drawn uniformly from 0 to 1, and its mean error is 25.5 percent, against 17.2 percent after training. The largest gain, 23.7 points on Vehicle, started from an error of 48.7 percent.

Two more contrasts were fixed before the run. Read by its class filters, the trained classifier still trails the tuned MLP. It is ahead on 3 datasets, significantly on none, and significantly behind on 4. Read instead by a nearest-neighbor vote, a single trained encoder with no hidden ranking layer comes close to ArrowFlow, with a mean error of 15.6 against 15.5 percent (Section S6.3).

Does ArrowFlow gain from it? We then rebuilt ArrowFlow for every split, with the configuration that ArrowFlow's own tuning chose on the inner folds of that split's training partition. In each view, an encoder network trained on the view's training rows replaced the fixed encoder (Section S6.4). This also removed the polynomial expansion and the three

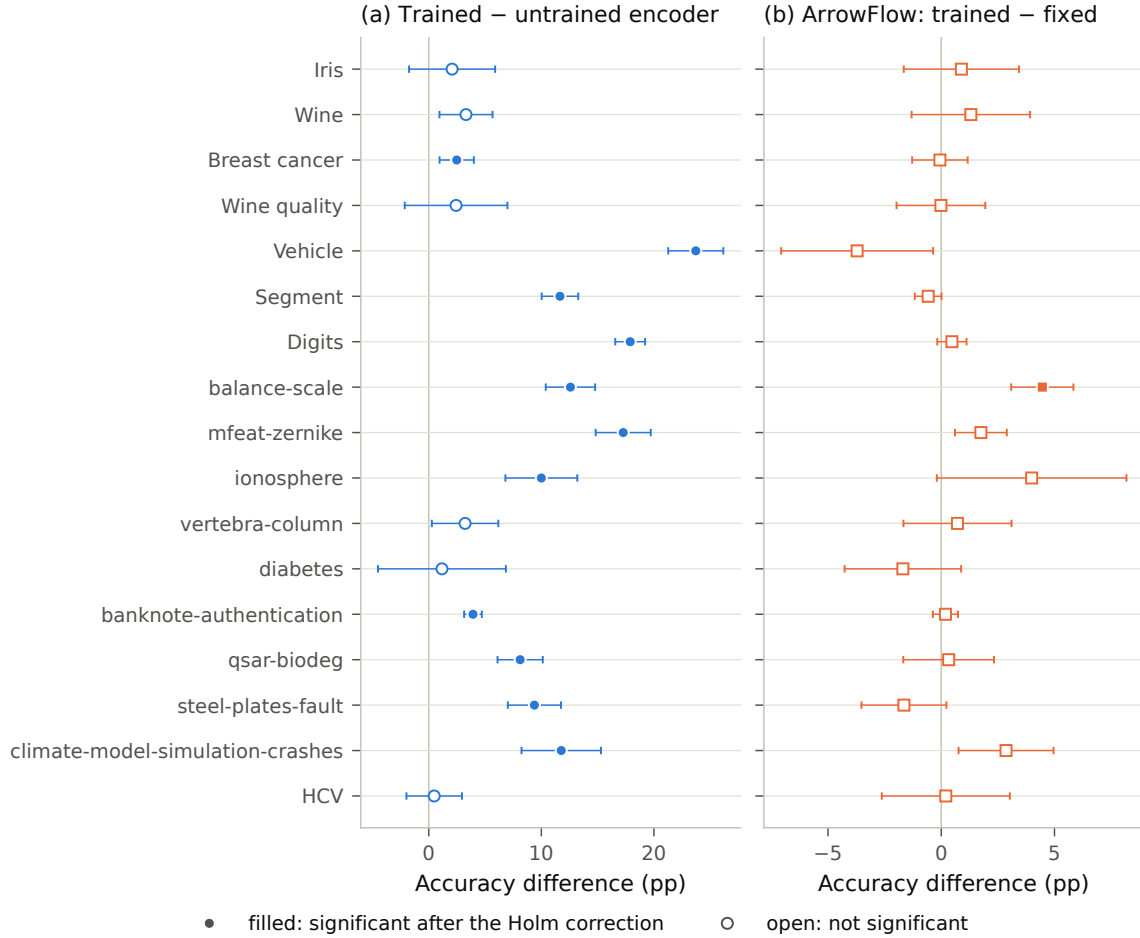


Figure 8: **Can the ranking signal train the encoder?** Accuracy differences, in percentage points, with 95% intervals (not simultaneous); positive favors the trained encoder. The two panels have different scales. The intervals are not adjusted for multiple comparisons, so an open mark’s interval can exclude zero. (a) The classifier of Section 5.5, an encoder network and one ranking layer of class filters, with the encoder trained by the converted motions, minus the same classifier with the encoder left at its random start. At that start, the weights of the encoder’s first two layers are drawn uniformly from 0 to 1. Training lowers the mean error over the datasets from 25.5 to 17.2 percent. (b) ArrowFlow with each view’s fixed encoder replaced by the trained encoder network, minus ArrowFlow with its fixed encoder. For every split, both use the configuration that ArrowFlow’s own tuning chose on the inner folds of that split’s training partition. The swap also drops the polynomial expansion and the projection strategies (Section S6.4). The mean error falls from 15.5 to 14.9 percent. Marks are circles in panel (a) and squares in panel (b). Filled marks remain significant after a Holm correction across the seventeen datasets: 11 in panel (a) and 1 in panel (b). The trained encoder has the higher mean accuracy on all 17 datasets in panel (a) and on 11 of the 17 in panel (b). The target conversion was developed on twelve of these datasets, all but Wine quality, banknote-authentication, qsar-biodeg, climate-model-simulation-crashes and HCV (Section S6.2). HCV: hepatitis C virus.

projection strategies. The views then differed only by their seeds, and no view used linear discriminant coordinates. With the fixed encoder, the rebuild reproduced the stored predictions exactly on the four test sets we checked, at all three fitting seeds (12 fits; Section S6.4).

With the trained encoder network, ArrowFlow had the higher mean accuracy on 11 of the 17 datasets, significantly on balance-scale, and was significantly less accurate on none (Figure 8b). Its mean error over the datasets fell from 15.5 to 14.9 percent. The rule fixed before the run has two conditions: a higher mean accuracy, and more datasets significantly higher than significantly lower. Both hold, so the rule reads that the trained encoder improves ArrowFlow. A single dataset, balance-scale, meets the second condition, and the conversion was developed partly on it. Across the datasets the gain is not established (post hoc one-sided signed-rank test, $p = 0.095$). Left untrained, the same network made ArrowFlow less accurate than the fixed encoder did on 16 of the 17 datasets. So the gain comes from training. Swapping in the network alone makes ArrowFlow worse.

How far does it go? The trained encoder made ArrowFlow less accurate on 6 of the 17 datasets, by up to 3.7 points on Vehicle, although none of these losses was significant. Every configuration had been tuned for the fixed encoder, which favors the fixed encoder in this comparison. With the trained encoder, ArrowFlow’s mean error of 14.9 percent is close to the tuned MLP’s 14.6. On balance-scale its error of 1.6 percent is the lowest of all the tuned models. But it still trails the best tuned classical model on 15 of the 17 datasets (Section S6.4). These comparisons are descriptive, and on balance-scale the lead over the MLP is not significant. The MLP reached its iteration cap before convergence on 331 of its 765 outer fits (Section S3.2). Against the single trained encoder read by a nearest-neighbor vote, ArrowFlow with the trained encoder is ahead on 15 of the 17 datasets, by 0.67 points on average, and significantly on none (Section S6.4).

8 Discussion

This paper set out to show that a network can compute with rankings and learn by counting votes. The experiments support that claim and one more. A learning rule that computes no derivative trains ranking layers, and the signal it sends down carries information. Motions of the same kind can also train a real-valued encoder network. But ArrowFlow trails the best tuned classical model on all seventeen small tabular datasets, and training its ranking layers adds only a modest gain. So the contribution is a construction and a learning rule inside permutation space, and the paper makes no accuracy argument.

What the training signal carries. The central finding is about the signal. Training helps because of the particular signal that the output layer returns, and matched controls separate that signal from mere movement of the filters (Section 7.5). If the update were only a useful disturbance, a scramble of the same size would cost nothing. Instead, a scrambled signal is worse than freezing the filters altogether, as a post hoc test across the datasets supports. And on most datasets it leaves an example’s nearest neighbors less likely to share its class. Both parts of the signal carry information: which filter a motion reaches, and which

way it points. Permuted alignment drew a new random assignment of motions to filters in every batch, and a fixed wrong assignment was not tested (Section S3.6).

What training adds. Training adds a modest gain, and an untrained network already does well. With the same readout, the untrained ArrowFlow comes within one percentage point of ArrowFlow on 12 of the 17 datasets. It is a permutation analog of random-weight models, which fix random hidden units and train only a readout [Pao et al., 1994, Huang et al., 2006]. It is also an analog of random features for kernel machines [Rahimi and Recht, 2007]. In the cited studies, such models generalized well on the benchmarks tested [Huang et al., 2006] and matched or outperformed kernel machines there [Huang et al., 2012, Rahimi and Recht, 2007]. The untrained ArrowFlow differs from them in two ways. Its units are permutations, and its readout is a nearest-neighbor vote rather than a least-squares linear map. Moreover, three of its seven views use the class labels, through a linear discriminant projection.

Read by a strong fixed readout, a Kendall-kernel classifier, the trained hidden rankings are no better than the untrained ones (Section 7.6). So training improves the neighborhoods that the nearest-neighbor readout uses. It does not improve the representation that a strong readout sees. Where the gain is large, it can sit in one place. On balance-scale, most of it comes from the rare class of level balances, which the support vector classifier recognizes far more often (Section 7.6). This locates the gain. It does not show that the network discovered the physical rule of the balance.

What the trained encoder adds. Motions can train more than ranking filters. Turned into target scores, they train the weights of an encoder network across a sort that is never differentiated. As a classifier, the trained encoder is more accurate than the same network untrained on every dataset, significantly on 11 (Section 7.7). The step is target propagation applied at a discrete interface [Bengio, 2014, Lee et al., 2015, Friesen and Domingos, 2018]. It does not relax the sort [Grover et al., 2019, Blondel et al., 2020b], and it does not copy a gradient across it [Bengio et al., 2013]. Still, with evenly spaced scores the step points the same way as a straight-through estimate for the same targets. The conversion multiplies each coordinate’s requested change of position by the gap between neighboring scores. A straight-through estimate that treats position as a linear function of score divides by that gap instead. Other methods also train through an exact discrete step (Section 2), among them blackbox differentiation [Vlastelica et al., 2020, Rolínek et al., 2020] and perturbed optimizers [Berthet et al., 2020]. This paper does not compare the conversion with them. Inside ArrowFlow the gain is modest and uneven. One dataset improves significantly and none worsens significantly, but six lose some accuracy. The comparison favors the fixed encoder, because every configuration was tuned for it. The encoder network is one small architecture, and the conversion and its settings were developed on single splits of twelve of the seventeen datasets before these runs (Section S6.2). The main method therefore keeps the fixed encoder. How well networks that mix ranking layers with ordinary layers can do when both are tuned together is still an open question.

What the Arrow lens delivers. The lens does two things. First, it locates context dependence. When the current order counts as a voter, no rule can avoid it. Consider any update rule in the setting of Proposition 1 that meets two mild conditions. It never reverses a pair on which the current order and all votes agree, and it neither freezes the filter nor always copies one vote. Every such rule is context dependent. Under the weight condition of Proposition 1, the weighted Borda count that updates a filter violates independence of irrelevant alternatives when the filter’s current order counts as a voter. An actual update holds that order fixed. In ArrowFlow’s layers it is then not Pareto efficient, so Arrow’s theorem does not cover it. A direct construction still shows a violation whenever its votes can move the filter (Section S1.1). A version of Arrow’s theorem without Pareto efficiency, Wilson’s theorem, implies the violation once the votes of a batch outweigh the current order [Wilson, 1972]. So a filter’s order of two items can depend on a third. The forward pass has no such dependence within a layer.

Second, the voting reading names three stages whose rules can be exchanged, although Arrow’s theorem speaks only to the update. The experiments exchanged all three, the view stage only with the prototype readout. The nearest-neighbor readout was more accurate than the prototype readout. Against a footrule median inside ArrowFlow’s own layers, the Borda count had the higher mean accuracy on 13 of the 17 datasets, significantly on one. Post hoc, the median learned little. On some datasets it barely moved the filters. On most others it moved them but ended within 0.2 points of untrained filters (Section S3.7). Across views, with the prototype readout, Borda and majority votes differ by at most 0.6 percentage points (Section S4). The lens predicted none of these outcomes. We claim no expressive power from the axiom violation, and the voting-rule study is not a test of Arrow’s theorem.

Boundaries. A second hidden layer did not improve accuracy at a matched configuration, with the earlier relay or with the corrected one. The rule passes supervision through several ranking layers, but the second layer brought no measured benefit. No experiment showed that the relayed signal improves the first layer. On the seven benchmark datasets, sorting the projected scores made no detectable difference to a nearest-neighbor readout, so there the ranking itself cannot be credited with any gain in accuracy. On the ten further datasets, where the comparison is descriptive, sorting raised the error on six, by up to 2.5 points (Section S3.4).

Limitations. Seventeen small tabular datasets are a narrow basis. The largest training partition has 1,848 rows (Segment). Fifteen test sets give wide intervals, and a difference that is not significant does not show that two models are equal. The candidate grid was small and anchored on default settings. A wider search inside the training data could move Table 3 in either direction. Settings were chosen by accuracy, and the readout does not weight classes. So a rare class can be neglected (Section 7.2). Each study’s own limits, such as single fitting seeds and unmatched architectures, are stated with its details in Sections S3 and S4.

9 Conclusion

We asked whether a network can compute with rankings and learn by counting votes, with no derivative computed. Each ranking layer orders its learned permutations by footrule distance to its input, and votes reorder the filters. The motions returned by the output layer’s votes train the last hidden layer, and every hidden layer above the first relays signed motions to the one below. Most experiments used an earlier version of this relay, which differs only in networks with two hidden layers (Section 3.5). A nearest-neighbor vote on the last hidden ranking predicts the class. Every filter update is an election decided by a weighted Borda count in which the filter’s current order also votes. Under a mild weight condition this rule violates independence of irrelevant alternatives by Arrow’s theorem (Section 4). An update that holds the current order fixed violates it too, by a direct construction, whenever its votes can move the filter.

Matched controls and a post hoc test across the datasets indicate that the returned motions carry information. Scrambling which filter a motion reaches, or which way it points, leaves the network less accurate than never moving the filters at all (Section 7.5). Motions of the same kind also train an ordinary neural network. Turned into target scores, they train an encoder network across a sort that is never differentiated. Swapped into ArrowFlow for its fixed encoder, the trained encoder raised the mean accuracy on 11 of the 17 datasets, significantly on one, and lowered it significantly on none (Section 7.7).

The limits are just as clear. With every configuration chosen inside the training data, ArrowFlow trails the best tuned classical model on all seventeen tabular datasets (Table 3). Its average rank, fourth of six, lies between random forest and gradient boosting; on the ten further datasets alone it is fifth. So this paper offers a learning rule inside permutation space, and no gain in accuracy (Section 7.6). A second hidden layer did not improve accuracy.

Whether deeper hierarchies and larger datasets gain more from training remains open. How far networks that mix ranking layers with ordinary layers can go is still open.

Supplementary Materials

The supplementary material is a separate document with six sections: the proofs (Section S1), the method details, worked example and protocol (Section S2), the complete results (Section S3), the ablations (Section S4), the reproducibility records (Section S5) and the learned encoder (Section S6). It contains Tables S1 to S75 and Figures S1 to S5, with a table of contents and lists giving the titles of all tables and figures. The file is `supplementary_ArrowFlow_MAKE.pdf`. A separate code supplement, `ArrowFlow_code_MAKE.zip`, contains a source snapshot of the implementation, experiment modules and frozen protocol files, including the learned-encoder extension.

Author Contributions

Conceptualization, methodology, software, validation, formal analysis, investigation, data curation, writing (original draft preparation, review and editing) and visualization: O.Y. The author has read and agreed to the published version of the manuscript.

Funding

This research received no external funding.

Institutional Review Board Statement

Not applicable. The study analyzes only public, de-identified datasets.

Informed Consent Statement

Not applicable.

Data Availability Statement

The datasets are public. The seven benchmark datasets originate from the University of California, Irvine (UCI) Machine Learning Repository; we used the copies distributed with scikit-learn for Iris, Wine, Breast cancer and Digits and OpenML copies for Wine quality, Vehicle and Segment. The ten further datasets come from OpenML. Section S5.2 gives the source and content hash of every copy and the identifier of every OpenML copy, which names one fixed version; the protocols of the further datasets also pin their versions and file checksums (Section S3.1). The reference list cites each dataset.

The core implementation, the original experiment harness and its frozen protocol files are available at <https://github.com/yilmazozgur/arrowflow/tree/v2.0.0-make>, tag v2.0.0-make. A source snapshot that also includes the later learned-encoder experiments and the table and figure rendering scripts accompanies this submission as `ArrowFlow_code_MAKE.zip`. The archive records a checksum for each source file and explains the run records needed for reproduction. Section S5 lists every protocol file with its identifier, freeze time and code revision, and gives the environment and the commands that reproduce each family.

Every results table of this paper and its supplement, Figures 7 and 8 and Figures S4 and S5 are rendered by a script from the recorded run directories, and the script checks the numbers and counts that the text quotes from those runs. Tables 1 and 2, Tables S1, S2 and S4, Figures 1 to 6, Figures S1 to S3 and the worked example of Section S2.2 are written by hand; the test suite checks the worked example. The run directories and the rendering script are available from the author on request.

Acknowledgments

During the preparation of this work the author used Claude Opus 5, Claude Opus 5.5, Claude Fable 5 and Claude Fable 5.1 (Anthropic), and GPT 5.6 Sol and GPT 6 Astra (OpenAI) for software development, running and analysing the experiments, and editing the manuscript. The author reviewed all output and takes full responsibility for the content of the publication.

Conflicts of Interest

The author declares no conflicts of interest.

References

- Dimitris Achlioptas. Database-friendly random projections: Johnson-Lindenstrauss with binary coins. *Journal of Computer and System Sciences*, 66(4):671–687, 2003. doi: 10.1016/S0022-0000(03)00025-4.
- Stefan Aeberhard and M. Forina. Wine. UCI Machine Learning Repository, 1992. URL <https://doi.org/10.24432/C5PC7J>. Dataset.
- E. Alpaydin and C. Kaynak. Optical recognition of handwritten digits. UCI Machine Learning Repository, 1998. URL <https://doi.org/10.24432/C50P49>. Dataset; the study uses the scikit-learn subset.
- Giuseppe Amato and Pasquale Savino. Approximate similarity search in metric spaces using inverted files. In *Proceedings of the 3rd International ICST Conference on Scalable Information Systems (InfoScale 2008)*. ICST, 2008. doi: 10.4108/ICST.INFOSCALE2008.3486.
- Rosa I. Arriaga and Santosh Vempala. An algorithmic theory of learning: Robust concepts and random projection. *Machine Learning*, 63(2):161–182, 2006. doi: 10.1007/s10994-006-6265-7.
- Kenneth J. Arrow. *Social Choice and Individual Values*. John Wiley & Sons, 1951.
- Kenneth J. Arrow. *Social Choice and Individual Values*. Number 12 in Cowles Foundation Monograph. John Wiley & Sons, New York, second edition, 1963. URL <https://cowles.yale.edu/sites/default/files/2022-09/m12-2-all.pdf>.
- Guilherme Barreto and Ajalmar Neto. Vertebral column. UCI Machine Learning Repository, 2005. URL <https://doi.org/10.24432/C5K89B>. Dataset.
- Yoshua Bengio. How auto-encoders could provide credit assignment in deep networks via target propagation. *arXiv preprint arXiv:1407.7906*, 2014. URL <https://arxiv.org/abs/1407.7906>.
- Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013. URL <https://arxiv.org/abs/1308.3432>.
- Paul Bertens. Rank ordered autoencoders. *arXiv preprint arXiv:1605.01749*, 2016.
- Quentin Berthet, Mathieu Blondel, Olivier Teboul, Marco Cuturi, Jean-Philippe Vert, and Francis Bach. Learning with differentiable perturbed optimizers. In H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 9508–9519. Curran Associates, Inc., 2020. URL

https://proceedings.neurips.cc/paper_files/paper/2020/hash/6bb56208f672af0dd65451f869fedfd9-Abstract.html.

Bernd Bischl, Giuseppe Casalicchio, Matthias Feurer, Pieter Gijsbers, Frank Hutter, Michel Lang, Rafael Gomes Mantovani, Jan van Rijn, and Joaquin Vanschoren. OpenML benchmarking suites. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c7e1249ffc03eb9ded908c236bd1996d-Abstract-round2.html>.

Mathieu Blondel, André F. T. Martins, and Vlad Niculae. Learning with Fenchel-Young losses. *Journal of Machine Learning Research*, 21(35):1–69, 2020a. URL <https://jmlr.org/papers/v21/19-021.html>.

Mathieu Blondel, Olivier Teboul, Quentin Berthet, and Josip Djolonga. Fast differentiable sorting and ranking. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 950–959. PMLR, 2020b. URL <https://proceedings.mlr.press/v119/blondel20a.html>.

Lina Bonilla, Jacques Gautrais, Simon Thorpe, and Timothée Masquelier. Analyzing time-to-first-spike coding schemes: A theoretical approach. *Frontiers in Neuroscience*, 16: 971937, 2022. doi: 10.3389/fnins.2022.971937.

Remco R. Bouckaert and Eibe Frank. Evaluating the replicability of significance tests for comparing learning algorithms. In Honghua Dai, Ramakrishnan Srikant, and Chengqi Zhang, editors, *Advances in Knowledge Discovery and Data Mining (PAKDD 2004)*, volume 3056 of *Lecture Notes in Computer Science*, pages 3–12. Springer, 2004. doi: 10.1007/978-3-540-24775-3_3.

Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996. doi: 10.1007/BF00058655.

Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010. URL <https://www.microsoft.com/en-us/research/publication/from-ranknet-to-lambdarank-to-lambda-mart-an-overview/>.

Christopher J. C. Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. Learning to rank using gradient descent. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 89–96. ACM, 2005. doi: 10.1145/1102351.1102363.

Christopher J. C. Burges, Robert Ragno, and Quoc V. Le. Learning to rank with nonsmooth cost functions. In B. Schölkopf, J. Platt, and T. Hoffman, editors, *Advances in Neural Information Processing Systems*, volume 19. MIT Press, 2006. URL https://proceedings.neurips.cc/paper_files/paper/2006/hash/af44c4c56f385c43f2529f9b1b018f6a-Abstract.html.

- M. Buscema, S. Terzi, and W. Tastle. Steel plates faults. UCI Machine Learning Repository, 2010. URL <https://doi.org/10.24432/C5J88N>. Dataset.
- Timothy I. Cannings and Richard J. Samworth. Random-projection ensemble classification. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 79(4):959–1035, 2017. doi: 10.1111/rssb.12228.
- Ioannis Caragiannis, Ariel D. Procaccia, and Nisarg Shah. When do noisy votes reveal the truth? In *Proceedings of the Fourteenth ACM Conference on Electronic Commerce (EC ’13)*, pages 143–160. ACM, 2013. doi: 10.1145/2482540.2482570.
- Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010. URL <https://www.jmlr.org/papers/v11/cawley10a.html>.
- Moses S. Charikar. Similarity estimation techniques from rounding algorithms. In *Proceedings of the 34th Annual ACM Symposium on Theory of Computing*, pages 380–388. ACM, 2002. doi: 10.1145/509907.509965.
- Edgar Chávez, Karina Figueroa, and Gonzalo Navarro. Effective proximity retrieval by ordering permutations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 30(9):1647–1658, 2008. doi: 10.1109/TPAMI.2007.70815.
- Youngmin Cho and Lawrence K. Saul. Kernel methods for deep learning. In Y. Bengio, D. Schuurmans, J. Lafferty, C. Williams, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 22. Curran Associates, 2009. URL https://proceedings.neurips.cc/paper_files/paper/2009/hash/5751ec3e9a4feab575962e78e006250d-Abstract.html.
- Michael Collins. Discriminative training methods for hidden Markov models: Theory and experiments with perceptron algorithms. In *Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing (EMNLP 2002)*, pages 1–8. Association for Computational Linguistics, 2002. doi: 10.3115/1118693.1118694.
- Vincent Conitzer and Tuomas Sandholm. Common voting rules as maximum likelihood estimators. In *Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence*, pages 145–152, 2005.
- W. J. Conover and Ronald L. Iman. Rank transformations as a bridge between parametric and nonparametric statistics. *The American Statistician*, 35(3):124–129, 1981. doi: 10.1080/00031305.1981.10479327.
- Paulo Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis. Wine quality. UCI Machine Learning Repository, 2009. URL <https://doi.org/10.24432/C56S3T>. Dataset; red-wine subset.
- Thomas M. Cover. The number of linearly inducible orderings of points in d -space. *SIAM Journal on Applied Mathematics*, 15(2):434–439, 1967. doi: 10.1137/0115039.

- Marco Cuturi, Olivier Teboul, and Jean-Philippe Vert. Differentiable ranking and sorting using optimal transport. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/hash/d8c24ca8f23c562a5600876ca2a550ce-Abstract.html.
- Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7(1):1–30, 2006. URL <https://www.jmlr.org/papers/v7/demsar06a.html>.
- Persi Diaconis and R. L. Graham. Spearman’s footrule as a measure of disarray. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(2):262–268, 1977. doi: 10.1111/j.2517-6161.1977.tb01624.x.
- Cameron Diao, Denis Kleyko, Jan M. Rabaey, and Bruno A. Olshausen. Generalized learning vector quantization for classification in randomized neural networks and hyperdimensional computing. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pages 1–9. IEEE, 2021. doi: 10.1109/IJCNN52387.2021.9533316.
- Thomas G. Dietterich. Ensemble methods in machine learning. In Josef Kittler and Fabio Roli, editors, *Multiple Classifier Systems*, volume 1857 of *Lecture Notes in Computer Science*, pages 1–15. Springer, 2000. URL https://doi.org/10.1007/3-540-45014-9_1.
- Dheeru Dua and Casey Graff. UCI machine learning repository. <https://archive.ics.uci.edu>, 2019.
- Shijin Duan, Yejia Liu, Shaolei Ren, and Xiaolin Xu. LeHDC: Learning-based hyperdimensional computing classifier. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*, pages 1111–1116. ACM, 2022. doi: 10.1145/3489517.3530593.
- Robert Duin. Multiple features. UCI Machine Learning Repository, 1998. URL <https://doi.org/10.24432/C5HC70>. Dataset; the study uses the Zernike-moment features.
- Cynthia Dwork, Ravi Kumar, Moni Naor, and D. Sivakumar. Rank aggregation methods for the web. In *Proceedings of the 10th International Conference on World Wide Web*, pages 613–622. ACM, 2001. doi: 10.1145/371920.372165.
- Kave Eshghi and Shyamsundar Rajaram. Locality sensitive hash functions based on concomitant rank order statistics. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 221–229. ACM, 2008. doi: 10.1145/1401890.1401921.
- Ronald Fagin, Ravi Kumar, and D. Sivakumar. Comparing top k lists. *SIAM Journal on Discrete Mathematics*, 17(1):134–160, 2003. doi: 10.1137/S0895480102412856.
- Ronald Fagin, Ravi Kumar, Mohammad Mahdian, D. Sivakumar, and Erik Vee. Comparing partial rankings. *SIAM Journal on Discrete Mathematics*, 20(3):628–648, 2006. doi: 10.1137/05063088X.

- Manuel Fernández-Delgado, Eva Cernadas, Senén Barro, and Dinani Amorim. Do we need hundreds of classifiers to solve real world classification problems? *Journal of Machine Learning Research*, 15(90):3133–3181, 2014. URL <https://jmlr.org/papers/v15/delgado14a.html>.
- R. A. Fisher. Iris. UCI Machine Learning Repository, 1936. URL <https://doi.org/10.24432/C56C76>. Dataset.
- Michael A. Fligner and Joseph S. Verducci. Distance based ranking models. *Journal of the Royal Statistical Society: Series B (Methodological)*, 48(3):359–369, 1986. doi: 10.1111/j.2517-6161.1986.tb01420.x.
- Abram L. Friesen and Pedro Domingos. Deep learning as a mixed convex-combinatorial optimization problem. In *International Conference on Learning Representations (ICLR)*, 2018. URL <https://openreview.net/forum?id=B1Lc-Gb0Z>. arXiv:1710.11573.
- Jacques Gautrais and Simon Thorpe. Rate coding versus temporal order coding: a theoretical approach. *BioSystems*, 48(1–3):57–65, 1998. doi: 10.1016/S0303-2647(98)00050-1.
- Donald Geman, Christian d’Avignon, Daniel Q. Naiman, and Raimond L. Winslow. Classifying gene expression profiles from pairwise mRNA comparisons. *Statistical Applications in Genetics and Molecular Biology*, 3(1):Article 19, 2004. doi: 10.2202/1544-6115.1071.
- Jacob Goldberger, Sam Roweis, Geoffrey Hinton, and Ruslan Salakhutdinov. Neighbourhood components analysis. In *Advances in Neural Information Processing Systems*, volume 17, pages 513–520, 2004. URL https://papers.nips.cc/paper_files/paper/2004/hash/42fe880812925e520249e808937738d2-Abstract.html.
- Stephen Grossberg. Competitive learning: From interactive activation to adaptive resonance. *Cognitive Science*, 11(1):23–63, 1987. doi: 10.1111/j.1551-6708.1987.tb00862.x.
- Aditya Grover, Eric Wang, Aaron Zweig, and Stefano Ermon. Stochastic optimization of sorting networks via continuous relaxations. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=H1eSS3CcKX>.
- Barbara Hammer, Daniela Hofmann, Frank-Michael Schleif, and Xibin Zhu. Learning vector quantization for (dis-)similarities. *Neurocomputing*, 131:43–51, 2014. doi: 10.1016/j.neucom.2013.05.054.
- Geoffrey Hinton. The forward-forward algorithm: Some preliminary investigations. *arXiv preprint arXiv:2212.13345*, 2022.
- Tin Kam Ho, J. J. Hull, and S. N. Srihari. Decision combination in multiple classifier systems. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16(1):66–75, 1994. doi: 10.1109/34.273716.
- Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL <https://www.jstor.org/stable/4615733>.

- Guang-Bin Huang, Qin-Yu Zhu, and Chee-Kheong Siew. Extreme learning machine: Theory and applications. *Neurocomputing*, 70(1–3):489–501, 2006. doi: 10.1016/j.neucom.2005.12.126.
- Guang-Bin Huang, Hongming Zhou, Xiaojian Ding, and Rui Zhang. Extreme learning machine for regression and multiclass classification. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 42(2):513–529, 2012. doi: 10.1109/TSMCB.2011.2168604.
- Image Segmentation. UCI Machine Learning Repository, 1990. URL <https://doi.org/10.24432/C5GP4N>. Dataset.
- Yunlong Jiao and Jean-Philippe Vert. The Kendall and Mallows kernels for permutations. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1935–1944. PMLR, 2015. URL <https://proceedings.mlr.press/v37/jiao15.html>.
- Yunlong Jiao and Jean-Philippe Vert. The Kendall and Mallows kernels for permutations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(7):1755–1769, 2018. doi: 10.1109/TPAMI.2017.2719680.
- William B. Johnson and Joram Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984. doi: 10.1090/conm/026/737400.
- Sanaa Kamal, Mohamed ElEleimy, Doaa Hegazy, and Mahmoud Nasr. Hepatitis C virus (HCV) for Egyptian patients. UCI Machine Learning Repository, 2017. URL <https://doi.org/10.24432/C5989V>. Dataset; the study uses the OpenML copy, data ID 46850.
- Pentti Kanerva. Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. *Cognitive Computation*, 1(2):139–159, 2009. doi: 10.1007/s12559-009-9009-8.
- John G. Kemeny. Mathematics without numbers. *Daedalus*, 88(4):577–591, 1959. URL <https://www.jstor.org/stable/20026529>.
- M. G. Kendall. A new measure of rank correlation. *Biometrika*, 30(1–2):81–93, 1938. doi: 10.1093/biomet/30.1-2.81.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- Peter Kirby, Laura Smets, Werner Van Leekwijck, and José Oramas. Simple encoder training for hyperdimensional computing. *Neural Computation*, 38(10):1829–1849, 2026. doi: 10.1162/neco.a.1567. URL <https://doi.org/10.1162/neco.a.1567>.

- Denis Kleyko, Dmitri A. Rachkovskij, Evgeny Osipov, and Abbas Rahimi. A survey on hyperdimensional computing aka vector symbolic architectures, part I: Models and data transformations. *ACM Computing Surveys*, 55(6):130:1–130:40, 2022. doi: 10.1145/3538531.
- Denis Kleyko, Dmitri A. Rachkovskij, Evgeny Osipov, and Abbas Rahimi. A survey on hyperdimensional computing aka vector symbolic architectures, part II: Applications, cognitive models, and challenges. *ACM Computing Surveys*, 55(9):175:1–175:52, 2023. doi: 10.1145/3558000.
- Teuvo Kohonen. The self-organizing map. *Proceedings of the IEEE*, 78(9):1464–1480, 1990. doi: 10.1109/5.58325.
- Ludmila I. Kuncheva and Christopher J. Whitaker. Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy. *Machine Learning*, 51(2):181–207, 2003. doi: 10.1023/A:1022859003006.
- Dong-Hyun Lee, Saizheng Zhang, Asja Fischer, and Yoshua Bengio. Difference target propagation. In Annalisa Appice, Pedro Pereira Rodrigues, Vítor Santos Costa, Carlos Soares, João Gama, and Alípio Jorge, editors, *Machine Learning and Knowledge Discovery in Databases*, volume 9284 of *Lecture Notes in Computer Science*, pages 498–515. Springer, 2015. URL https://doi.org/10.1007/978-3-319-23528-8_31.
- Timothy P. Lillicrap, Daniel Counden, Douglas B. Tweed, and Colin J. Akerman. Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, 7:13276, 2016. doi: 10.1038/ncomms13276.
- Volker Lohweg. Banknote authentication. UCI Machine Learning Repository, 2012. URL <https://doi.org/10.24432/C55P57>. Dataset.
- D. Lucas, R. Klein, J. Tannahill, D. Ivanova, S. Brandon, D. Domyancic, and Y. Zhang. Climate model simulation crashes. UCI Machine Learning Repository, 2013. URL <https://doi.org/10.24432/C5HG71>. Dataset.
- Colin L. Mallows. Non-null ranking models. I. *Biometrika*, 44(1–2):114–130, 1957. doi: 10.1093/biomet/44.1-2.114.
- Kamel Mansouri, Tine Ringsted, Davide Ballabio, Roberto Todeschini, and Viviana Consonni. QSAR biodegradation. UCI Machine Learning Repository, 2013. URL <https://doi.org/10.24432/C5H60M>. Dataset.
- Thomas M. Martinetz, Stanislav G. Berkovich, and Klaus J. Schulten. “Neural-Gas” network for vector quantization and its application to time-series prediction. *IEEE Transactions on Neural Networks*, 4(4):558–569, 1993. doi: 10.1109/72.238311.
- Eric Maskin. Borda’s rule and Arrow’s independence condition. *Journal of Political Economy*, 133(2):385–420, 2025. doi: 10.1086/732892.

- Gonzalo Mena, David Belanger, Scott Linderman, and Jasper Snoek. Learning latent permutations with Gumbel–Sinkhorn networks. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=Byt3oJ-0W>.
- Yair Movshovitz-Attias, Alexander Toshev, Thomas K. Leung, Sergey Ioffe, and Saurabh Singh. No fuss distance metric learning using proxies. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 360–368. IEEE, 2017. doi: 10.1109/ICCV.2017.47.
- Pete Mowforth and Barry Shepherd. Statlog (vehicle silhouettes). UCI Machine Learning Repository, n.d. URL <https://doi.org/10.24432/C5HG6N>. Dataset; accessed 11 September 2026.
- Claude Nadeau and Yoshua Bengio. Inference for the generalization error. *Machine Learning*, 52(3):239–281, 2003. doi: 10.1023/A:1024068626366.
- David Nebel, Barbara Hammer, Kathleen Frohberg, and Thomas Villmann. Median variants of learning vector quantization for learning of dissimilarity data. *Neurocomputing*, 169: 295–305, 2015. doi: 10.1016/j.neucom.2014.12.096.
- Mathias Niepert, Pasquale Minervini, and Luca Franceschi. Implicit MLE: Backpropagating through discrete exponential family distributions. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 14567–14579. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/7a430339c10c642c4b2251756fd1b484-Abstract.html.
- Yoh-Han Pao, Gwang-Hoon Park, and Dejan J. Sobajic. Learning and generalization characteristics of the random vector functional-link net. *Neurocomputing*, 6(2):163–180, 1994. doi: 10.1016/0925-2312(94)90053-1.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Elżbieta Pełkalska and Robert P. W. Duin. *The Dissimilarity Representation for Pattern Recognition: Foundations and Applications*. Series in Machine Perception and Artificial Intelligence. World Scientific, Singapore, 2005. doi: 10.1142/5965.
- Tony A. Plate. Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6(3):623–641, 1995. doi: 10.1109/72.377968.
- Sebastian Prillo and Julian Eisenschlos. SoftSort: A continuous relaxation for the argsort operator. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning*

- Research*, pages 7793–7802. PMLR, 2020. URL <https://proceedings.mlr.press/v119/prillo20a.html>.
- Abbas Rahimi, Pentti Kanerva, Luca Benini, and Jan M. Rabaey. Efficient biosignal processing using hyperdimensional computing: Network templates for combined learning and classification of ExG signals. *Proceedings of the IEEE*, 107(1):123–143, 2019. doi: 10.1109/JPROC.2018.2871163.
- Ali Rahimi and Benjamin Recht. Random features for large-scale kernel machines. In J. Platt, D. Koller, Y. Singer, and S. Roweis, editors, *Advances in Neural Information Processing Systems*, volume 20, pages 1177–1184. Curran Associates, Inc., 2007. URL https://proceedings.neurips.cc/paper_files/paper/2007/hash/013a006f03dbc5392effeb8f18fda755-Abstract.html.
- Michal Rolínek, Vít Musil, Anselm Paulus, Marin Vlastelica, Claudio Michaelis, and Georg Martius. Optimizing rank-based metrics with blackbox differentiation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7617–7627, 2020. doi: 10.1109/CVPR42600.2020.00764.
- Donald G. Saari. Connecting and resolving Sen’s and Arrow’s theorems. *Social Choice and Welfare*, 15(2):239–261, 1998. doi: 10.1007/s003550050103.
- Subham Sekhar Sahoo, Anselm Paulus, Marin Vlastelica, Vít Musil, Volodymyr Kuleshov, and Georg Martius. Backpropagation through combinatorial algorithms: Identity with projection works. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=JZMR727029>. arXiv:2205.15213.
- Atsushi Sato and Keiji Yamada. Generalized learning vector quantization. In D. Touretzky, M. C. Mozer, and M. Hasselmo, editors, *Advances in Neural Information Processing Systems*, volume 8, pages 423–429. MIT Press, 1995. URL https://proceedings.neurips.cc/paper_files/paper/1995/hash/9c3b1830513cc3b8fc4b76635d32e692-Abstract.html.
- Robert E. Schapire. The strength of weak learnability. *Machine Learning*, 5(2):197–227, 1990. doi: 10.1007/BF00116037.
- R. Siegler. Balance scale. UCI Machine Learning Repository, 1976. URL <https://doi.org/10.24432/C5488X>. Dataset.
- V. Sigillito, S. Wing, L. Hutton, and K. Baker. Ionosphere. UCI Machine Learning Repository, 1989. URL <https://doi.org/10.24432/C5W01B>. Dataset.
- Richard Sinkhorn and Paul Knopp. Concerning nonnegative matrices and doubly stochastic matrices. *Pacific Journal of Mathematics*, 21(2):343–348, 1967. doi: 10.2140/pjm.1967.21.343.

- Laura Smets, Dmitri Rachkovskij, Evgeny Osipov, Werner Van Leekwijck, Olexander Volkov, and Steven Latré. Margin-based training of HDC classifiers. *Big Data and Cognitive Computing*, 9(3):68, 2025. doi: 10.3390/bdcc9030068.
- Jack W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes. Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. In *Proceedings of the Annual Symposium on Computer Application in Medical Care*, pages 261–265, 1988. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC2245318/>.
- Simon Thorpe and Jacques Gautrais. Rapid visual processing using spike asynchrony. In M. C. Mozer, M. Jordan, and T. Petsche, editors, *Advances in Neural Information Processing Systems*, volume 9, pages 901–907. MIT Press, 1996. URL https://proceedings.neurips.cc/paper_files/paper/1996/hash/fd5c905bcd8c3348ad1b35d7231ee2b1-Abstract.html.
- Simon Thorpe, Arnaud Delorme, and Rufin Van Rullen. Spike-based strategies for rapid processing. *Neural Networks*, 14(6–7):715–725, 2001. doi: 10.1016/S0893-6080(01)00083-1.
- Merijn van Erp and Lambert Schomaker. Variants of the Borda count method for combining ranked classifier hypotheses. In Lambert Schomaker and Louis Vuurpijl, editors, *Proceedings of the 7th International Workshop on Frontiers in Handwriting Recognition (IWFHR)*, pages 443–452, Nijmegen, 2000. International Unipen Foundation.
- Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. OpenML: Networked science in machine learning. *ACM SIGKDD Explorations Newsletter*, 15(2):49–60, 2014. doi: 10.1145/2641190.2641198. URL <https://doi.org/10.1145/2641190.2641198>.
- Pere Vergés, Mike Heddes, Igor Nunes, Denis Kleyko, Tony Givargis, and Alexandru Nicolau. Classification using hyperdimensional computing: A review with comparative analysis. *Artificial Intelligence Review*, 58(6):173, 2025. doi: 10.1007/s10462-025-11181-2.
- Marin Vlastelica, Anselm Paulus, Vít Musil, Georg Martius, and Michal Rolínek. Differentiation of blackbox combinatorial solvers. In *International Conference on Learning Representations (ICLR)*, 2020. URL <https://arxiv.org/abs/1912.02175>. arXiv:1912.02175.
- Kilian Q. Weinberger and Lawrence K. Saul. Distance metric learning for large margin nearest neighbor classification. *Journal of Machine Learning Research*, 10(9):207–244, 2009. URL <https://www.jmlr.org/papers/v10/weinberger09a.html>.
- Robert Wilson. Social choice theory without the Pareto principle. *Journal of Economic Theory*, 5(3):478–486, 1972. doi: 10.1016/0022-0531(72)90051-8.
- William Wolberg, Olvi Mangasarian, Nick Street, and W. Street. Breast cancer Wisconsin (diagnostic). UCI Machine Learning Repository, 1993. URL <https://doi.org/10.24432/C5DW2B>. Dataset.
- Chang Xu, Dacheng Tao, and Chao Xu. A survey on multi-view learning. *arXiv preprint arXiv:1304.5634*, 2013.

- Jay Yagnik, Dennis Strelow, David A. Ross, and Ruei-Sung Lin. The power of comparative reasoning. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2431–2438. IEEE, 2011. doi: 10.1109/ICCV.2011.6126527.
- H. P. Young. An axiomatization of Borda’s rule. *Journal of Economic Theory*, 9(1):43–52, 1974. doi: 10.1016/0022-0531(74)90073-8.
- William S. Zwicker. Consistency without neutrality in voting rules: When is a vote an average? *Mathematical and Computer Modelling*, 48(9–10):1357–1373, 2008. doi: 10.1016/j.mcm.2008.05.031.